

Day 6

What is Machine Learning?

Big Picture • Concepts • Intuition • First ML Model

■ AI vs ML vs DL

■ Types of ML

■ ML Workflow

■ scikit-learn

■ Lab

1 Section 1 – AI vs ML vs Deep Learning

Before we write a single line of code, let's build a crystal-clear mental picture of what Artificial Intelligence, Machine Learning, and Deep Learning actually mean — and how they relate to each other.

The 'Russian Dolls' Analogy

Think of three Russian nesting dolls. The biggest doll is **AI**. Open it and you find **ML** inside. Open that and you find **Deep Learning** at the centre. Every DL system is also an ML system. Every ML system is also an AI system — but not the other way around.

■ AI (Artificial Intelligence)	■ ML (Machine Learning)	■ Deep Learning
The biggest umbrella. Any technique that makes a machine appear smart.	A subset of AI where machines learn patterns from data — no manual rules.	A subset of ML using layered neural networks inspired by the brain.
Chess engine, expert systems, voice assistants, self-driving cars	Spam filter, fraud detection, recommendation engines	Image recognition, ChatGPT, voice-to-text, style transfer

▲ Every column is a subset of the one on its left.

1A — Artificial Intelligence (AI)

AI is the broad science of creating machines that can perform tasks normally requiring human intelligence — reasoning, planning, understanding language, recognising faces, and so on.

■ Real-Life AI Examples You Already Know

- Chess computers (Deep Blue beat world champion in 1997)
- Voice assistants — Alexa, Siri, Google Assistant
- Self-driving cars that read road signs and avoid pedestrians
- Google Translate converting one language to another
- Your phone predicting the next word when you type

Early AI (before ~2000) was mostly **rule-based**: programmers hand-coded thousands of IF-THEN rules. E.g., IF email contains 'You have won a prize' THEN mark as spam. This works for simple cases but breaks when the world is complex and messy.

1B — Machine Learning (ML)

ML flips the programming model. Instead of humans writing the rules, we give the machine **lots of examples** and let it **discover the rules itself**.

■ The Key Shift — Old vs New Way

OLD WAY (Traditional Programming): Data + Rules → Answers

NEW WAY (Machine Learning): Data + Answers → Rules (Model)

Example: Spam detection. Old way — a developer writes: *if the word 'lottery' appears, mark as spam*. New way — show the machine 1 million emails labelled spam/not-spam and let it figure out the pattern. The ML model learns that spammers also use words like 'winner', 'claim', 'free gift', and many more — patterns a human might miss.

1C — Deep Learning (DL)

Deep Learning is ML with a special kind of model called a **Neural Network**. These networks are loosely inspired by the neurons in your brain. 'Deep' means the network has **many layers** — each layer learns increasingly abstract features.

■ How a Neural Network Sees a Cat Photo

Layer 1 — detects basic edges and corners in the pixels

Layer 2 — combines edges into shapes (circles, curves)

Layer 3 — combines shapes into parts (ears, eyes, whiskers)

Layer 4 — combines parts to recognise 'cat'

You don't program any of this — the network learns it from millions of cat photos!

Deep Learning powers ChatGPT, image recognition on your phone, YouTube's content recommendations, voice-to-text, and translation. Today we focus on regular ML — deep learning comes later in the course.

2 Section 2 – Types of Machine Learning

There are three main learning styles — think of them as three different kinds of school environments.

2A — Supervised Learning ■ (Learning with a Teacher)

In supervised learning, every training example comes with the correct answer (called a **label**). The model learns to map inputs to outputs by seeing thousands of labelled examples.

■ Classroom Analogy

Imagine a student learning to identify fruits. The teacher holds up an apple and says 'This is an apple'. Then a banana — 'This is a banana'. After seeing enough examples WITH labels, the student can identify new fruits independently. This is EXACTLY how supervised learning works!

Task	Input (X)	Output / Label (y)
Email spam filter	Email text	Spam / Not Spam
Predict house price	Size, location, rooms	Price in rupees
Medical diagnosis	X-ray image	Disease / No Disease
Student result	Study hours, attendance	Pass / Fail
Sentiment analysis	Review text	Positive / Negative

▲ Every row shows a supervised learning problem. The model trains on rows with known outputs.

Supervised learning has two sub-types:

■ **Regression:** predict a continuous number (price, temperature, score)

■■ **Classification:** predict a category (spam/not-spam, pass/fail, cat/dog)

2B — Unsupervised Learning ■ (Learning without a Teacher)

Here, the data has **no labels**. The machine must find hidden patterns, groups, or structures entirely on its own. There is no 'right answer' provided.

■■ Shopping Analogy

A new store owner doesn't know her customers yet. She looks at purchase histories and notices: Group A buys baby products, Group B buys sports gear, Group C buys luxury items. She never told the algorithm these groups — it discovered them! She can now target ads to each group. This is clustering — unsupervised learning.

Common unsupervised techniques:

■ **Clustering:** group similar data points (customer segmentation, document grouping)

■ **Dimensionality Reduction:** simplify data by keeping only important features

■ **Anomaly Detection:** find unusual data points (credit card fraud, defective products)

2C — Reinforcement Learning ■ (Learning by Trial, Reward & Punishment)

An agent learns by **interacting with an environment**. It tries actions, receives rewards for good actions and penalties for bad ones, and gradually learns the best strategy.

■ Video Game Analogy

Imagine teaching a robot to play a game — but you never explain the rules. The robot presses random buttons. When it scores a point → reward (+1 point). When it loses a life → penalty (-1 point). Over thousands of games it learns the perfect strategy entirely by trial and error. This is reinforcement learning!

Real-world applications: AlphaGo (beat world champion at Go), self-driving car simulation training, robot arm learning to grasp objects, personalised recommendation systems, algorithmic trading.

Feature	Supervised	Unsupervised	Reinforcement
Labels needed?	■ Yes	■ No	■ Rewards only
Teacher present?	■ Yes	■ No	Indirect (environment)
Goal	Predict label	Find structure	Maximise reward

Difficulty (beginner)	■■ Easy	■■■■ Medium	■■■■■ Hard
Today's focus?	■ YES	Later in course	Later in course

▲ Quick comparison of the three ML types.

3 Section 3 – The ML Workflow (5 Steps)

Every ML project — from predicting house prices to detecting cancer — follows the same 5-step pipeline. Master this pipeline and you understand 80% of what a data scientist does.

Collect and Understand Data

Gather the raw data you need. Understand what each column means.

1

Ask: Do we have enough data? Is it relevant? Where did it come from?

Example: Collect 1,000 students' records — hours studied & pass/fail result.

Clean and Prepare the Data

Real data is messy! Fix missing values, remove duplicates, fix typos.

2

Feature engineering: create new useful columns from existing ones.

Split data into Training set (80%) and Testing set (20%).

Train a Model

Choose an algorithm (e.g., Logistic Regression, Decision Tree).

3

Feed the training data into the algorithm using `fit()`.

The model adjusts its internal parameters to learn the pattern.

Evaluate — Is It Accurate?

Test the trained model on the test set (data it has NEVER seen).

4

Measure accuracy, precision, recall depending on the problem type.

If accuracy is low, go back and improve data or try a different algorithm.

Deploy — Use It in the Real World

Package the model into an app, API, or website.

5

Monitor performance over time — the world changes, so must the model.

Example: Deploy as a web app where students enter hours and get prediction.

■ The 'Garbage In, Garbage Out' Rule

The single most important factor in ML success is DATA QUALITY.

A brilliant algorithm on bad data gives terrible results.

A simple algorithm on great data often beats a complex one on bad data.

Steps 1 and 2 typically consume 70-80% of a data scientist's time!

4 Section 4 – Introduction to scikit-learn (sklearn)

scikit-learn (imported as **sklearn**) is the most popular and beginner-friendly Machine Learning library for Python. It was created in 2007 and is now used by thousands of companies and researchers worldwide.

■ Why scikit-learn?

- 100+ ML algorithms all accessible with the SAME simple API
- Works seamlessly with NumPy and pandas
- Excellent documentation with examples for every function
- Free, open-source, and battle-tested in production
- Used at Spotify, Booking.com, JP Morgan, and many more

The Three Magic Methods — The Whole sklearn API in 3 Words

No matter which algorithm you use — whether it's a simple linear model or a complex random forest — sklearn always provides the same three methods:

Method	What It Does	Analogy
.fit(X, y)	Train the model on your data	Student studies textbook
.predict(X)	Make predictions on new data	Student takes the exam
.score(X, y)	Measure accuracy (0.0 to 1.0)	Teacher grades the exam

▲ Learn these 3 methods and you can use ANY sklearn algorithm immediately.

Installing and Importing sklearn

```
# Install (run once in terminal):
pip install scikit-learn

# Basic imports you will use every session:
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# sklearn also includes toy datasets for practice:
from sklearn.datasets import load_iris, load_digits
```

The Universal sklearn Pattern (Template)

Every sklearn project follows this exact skeleton — copy-paste it for any problem:

```
# STEP 1: Import
from sklearn.some_module import SomeAlgorithm
from sklearn.model_selection import train_test_split

# STEP 2: Prepare Data
X = feature_data          # Input columns (2D array / DataFrame)
y = labels                # Target column (1D array / Series)

# STEP 3: Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# STEP 4: Train Model
model = SomeAlgorithm()   # Create an instance of the algorithm
model.fit(X_train, y_train) # Train on training data

# STEP 5: Predict and Evaluate
predictions = model.predict(X_test) # Predict on test data
accuracy    = model.score(X_test, y_test) # How accurate?
print(f'Accuracy: {accuracy:.2%}')      # e.g., Accuracy: 92.50%
```

5 Section 5 – LAB: Predict Student Pass / Fail

Time to build your **very first Machine Learning model**! We will predict whether a student passes or fails based on how many hours they studied. This is a **supervised classification** problem.

■ Lab Objectives

1. Understand the full ML workflow end-to-end (all 5 steps)
2. Create a dataset manually and load it with pandas
3. Train a Logistic Regression classifier using sklearn
4. Evaluate accuracy and interpret results
5. Make predictions for new students

Step 1 — Understand the Problem

Question: Given the number of hours a student studies, can we predict whether they will Pass (1) or Fail (0)?

Input (Feature X): Hours Studied (e.g., 1, 3, 5, 7)

Output (Label y): Pass=1, Fail=0

Our Training Data:

Student	Hours Studied	Result (Label)
Student 1	1 hour	Fail (0)

Student 2	2 hours	Fail (0)
Student 3	3 hours	Fail (0)
Student 4	4 hours	Pass (1)
Student 5	5 hours	Pass (1)
Student 6	6 hours	Pass (1)
Student 7	7 hours	Pass (1)
Student 8	8 hours	Pass (1)
Student 9	9 hours	Pass (1)
Student 10	10 hours	Pass (1)

▲ Our tiny labelled dataset. Students studying ≥ 4 hours pass in this example.

Step 2 — Full Lab Code (Complete, Runnable)

Copy this exactly into a file called `day6_lab.py` and run it:

[illegible]

```
# ■■■ Part 2
```

```
# ■■ 4. Create and Train the Model ■■■  
model = LogisticRegression()  
model.fit(X_train, y_train)    # THE magic line – learning happens!  
print('Model trained successfully!')
```

```
# ■■ 5. Evaluate on Test Set ■■■  
y_pred = model.predict(X_test)  
accuracy = accuracy_score(y_test, y_pred)  
print(f'Accuracy: {accuracy:.2%}')  
print(classification_report(y_test, y_pred,  
                             target_names=['Fail', 'Pass']))
```

```
# ■■ 6. Predict for Brand-New Students ■■■  
new_students = [[2], [5], [8]]  
predictions = model.predict(new_students)  
labels = {0: 'FAIL', 1: 'PASS'}  
for hrs, pred in zip(new_students, predictions):  
    print(f'Studied {hrs[0]} hrs → {labels[pred]}')
```

Step 3 — Understanding Every Line

Code	What it does / Why
<code>import numpy as np</code>	NumPy handles numerical arrays. sklearn works on arrays.
<code>import pandas as pd</code>	Pandas creates DataFrames — like Excel tables in Python.
<code>from sklearn.linear_model import LogisticRegression</code>	Import the classification algorithm we will use.
<code>df = pd.DataFrame(data)</code>	Convert our Python dictionary into a proper table.
<code>X = df[['hours_studied']]</code>	X = inputs. Double brackets <code>[[]]</code> give us a 2D array (required by sklearn).
<code>y = df['pass_fail']</code>	y = labels. Single bracket gives 1D array (correct for labels).
<code>train_test_split(... test_size=0.2)</code>	Randomly split data: 80% to train, 20% to test. Never train on test data!
<code>random_state=42</code>	Fixes the random seed so the split is reproducible (same every run).
<code>model = LogisticRegression()</code>	Create an untrained model object. Think of it as a blank brain.
<code>model.fit(X_train, y_train)</code>	THE most important line. The model learns patterns from training data.
<code>model.predict(X_test)</code>	Apply learned patterns to new/unseen data to generate predictions.
<code>accuracy_score(y_test, y_pred)</code>	Compare predictions to real answers. Returns a number 0.0 to 1.0.
<code>classification_report(...)</code>	Detailed breakdown: precision, recall, F1 per class.
<code>model.predict([[2],[5],[8]])</code>	Predict pass/fail for students who studied 2, 5, and 8 hours.

Step 4 — Expected Output

Our Dataset:

	hours_studied	pass_fail
0	1	0
1	2	0
2	3	0
3	4	1
...	...	...

Training samples: 8, Test samples: 2

Model has been trained!

Accuracy: 100.00%

	precision	recall	f1-score	support
Fail	1.00	1.00	1.00	1
Pass	1.00	1.00	1.00	1
accuracy			1.00	2

Student who studied 2 hrs: FAIL

Student who studied 5 hrs: PASS

Student who studied 8 hrs: PASS

■ Why 100% Accuracy Here?

With only 10 data points and a clear pattern (≥ 4 hrs = pass), the model learns perfectly. In real life, data is noisy and 100% accuracy is suspicious! A 85-95% accuracy on a real dataset would be excellent.

6 Section 6 – Challenge Exercises

Complete these exercises to solidify your Day 6 understanding:

■ **Easy:** Add a 2nd feature — 'attendance_percent' — and retrain.

■ **Easy:** Change test_size to 0.3. Does accuracy change? Why?

■ **Medium:** Replace LogisticRegression with DecisionTreeClassifier and compare.

■ **Medium:** Add 20 more rows of data with realistic noise. What happens to accuracy?

■ **Hard:** Add 'sleep_hours' and 'previous_score'. Measure which feature matters most.

■ **Hard:** Find a real dataset on Kaggle, apply the same 5-step workflow.

7 Section 7 – Key Concepts Glossary

Term	Simple Definition
Algorithm	A step-by-step method a machine uses to learn from data.
Model	The output of training — the 'brain' that makes predictions.
Feature (X)	An input variable used to make predictions (e.g., hours studied).
Label (y)	The correct answer / output variable (e.g., pass/fail).
Training Set	Data the model learns from (typically 80% of your data).
Test Set	Data the model is evaluated on — it must NEVER see this during training.
Accuracy	Fraction of correct predictions. 0.90 = 90% correct.
fit()	Train the model on data. The learning happens here.
predict()	Use the trained model to make predictions on new data.
score()	Returns accuracy — how well the model performs on given data.
Overfitting	Model memorises training data but fails on new data (too specific).
Underfitting	Model is too simple to capture the pattern (too general).
Classification	Predicting a category/class (spam/not-spam, pass/fail).
Regression	Predicting a continuous number (house price, temperature).
Logistic Regression	Despite the name, it is a CLASSIFICATION algorithm — not regression!

8 Section 8 – Day 6 Summary & What's Next

Today You Mastered:

- **AI vs ML vs DL:** nested concepts — $DL \subset ML \subset AI$
- **Supervised Learning:** labelled data → learn to predict
- **Unsupervised Learning:** no labels → find hidden patterns
- **Reinforcement Learning:** reward/punishment → optimal strategy
- **ML Workflow:** 5 steps — Collect → Clean → Train → Evaluate → Deploy
- **scikit-learn:** fit(), predict(), score() — the universal API
- **First ML Model:** Logistic Regression to predict student pass/fail

■ Coming Up — Day 7 Preview

Day 7: Linear Regression — Predicting Continuous Numbers

- Predict house prices, student scores, sales revenue
- Understand the maths behind the straight line ($y = mx + c$)
- Visualise regression with matplotlib
- Lab: Predict salary from years of experience

Remember the Golden Rule of ML:

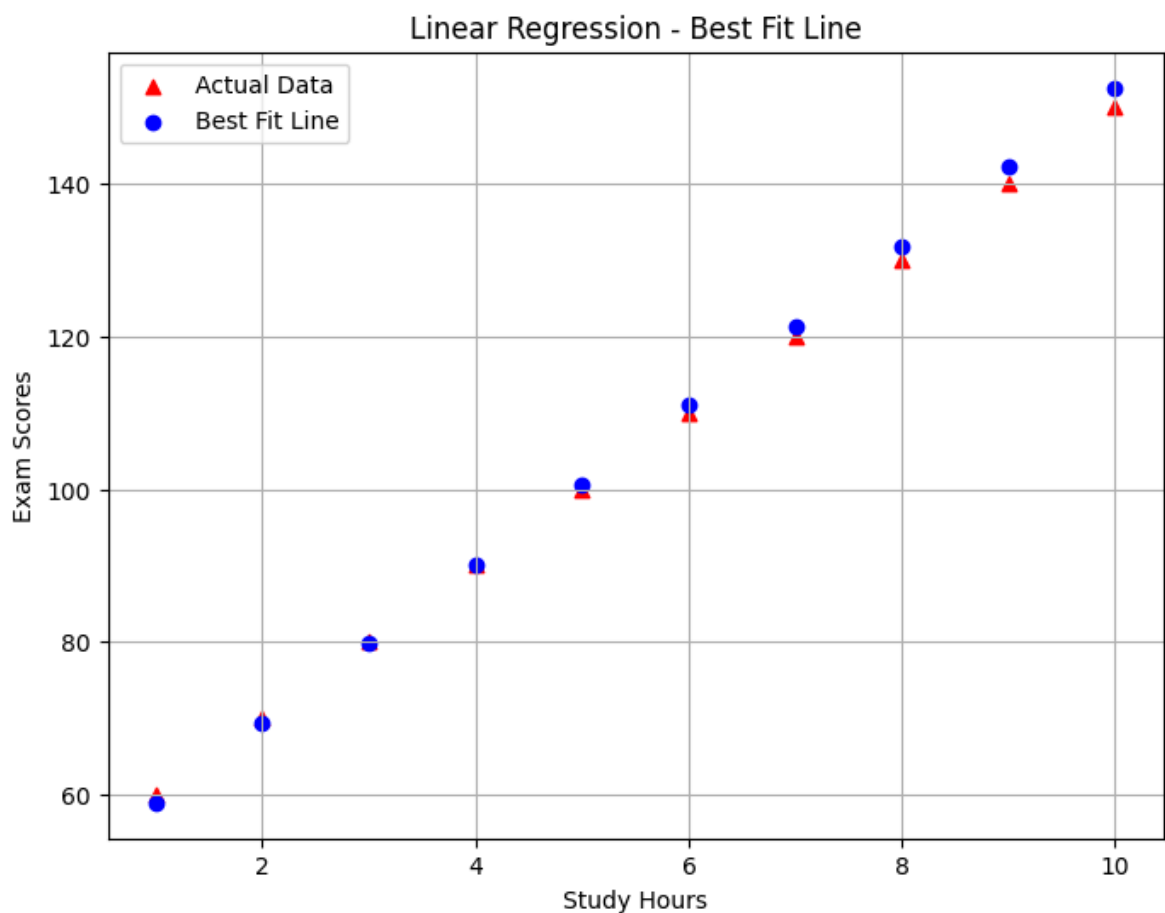
"More good data always beats a more complex algorithm. Understand your data first, model second." — Andrew Ng



Day 6 Material — Beginner ML Course • 6 Hours • No Maths — Only Intuition!

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import numpy as np
import matplotlib.pyplot as plt
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])
predicted_scores = 10.41* hours + 48.52
plt.figure(figsize=(8,6))
plt.scatter(hours, scores, label="Actual Data" ,marker="^",color="red")
plt.scatter(hours, predicted_scores, label="Best Fit Line",color="blue")
plt.xlabel("Study Hours")
plt.ylabel("Exam Scores")
plt.title("Linear Regression - Best Fit Line")
plt.legend()
plt.grid(True)
plt.show()
```



```
x=np.array([1,2,3,4,5],dtype=float)
y=np.array([60,70,80,90,100],dtype=float)
m=0
c=0
learning_rate=0.01
epochs=1000
n=len(x)
```

```

for i in range(epochs):
    y_pred=m*x+c
    error=y-y_pred
    mse=np.mean(error**2)
    dm=(-2/n)*np.sum(x*error)
    dc=(-2/n)*np.sum(error)
    m=m-learning_rate*dm
    c=c-learning_rate*dc
    if i%100==0:
        print(f"Epoch{i}")
        print(f"MSE={mse:.2f}")
        print(f"m={m:.2f},c={c:.2f}")
        print("-----")
print("\nFinal Best Fit Line:")
print(f"y={m:.2f}x+{c:.2f}")

```

```

Epoch0
MSE=6600.00
m=5.20,c=1.60
-----
Epoch100
MSE=177.90
m=18.63,c=18.84
-----
Epoch200
MSE=90.37
m=16.15,c=27.79
-----
Epoch300
MSE=45.90
m=14.38,c=34.17
-----
Epoch400
MSE=23.32
m=13.12,c=38.72
-----
Epoch500
MSE=11.84
m=12.23,c=41.96
-----
Epoch600
MSE=6.02
m=11.59,c=44.27
-----
Epoch700
MSE=3.06
m=11.13,c=45.92
-----
Epoch800
MSE=1.55
m=10.81,c=47.09
-----
Epoch900
MSE=0.79
m=10.57,c=47.93
-----

Final Best Fit Line:
y=10.41x+48.52

```

```
from sklearn.model_selection import train_test_split
import numpy as np
```

```
experience=np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary=np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])
x=experience.reshape(-1,1)
y=salary
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_stat
print(f"Training samples:{len(x_train)}")
print(f"Testing samples:{len(x_test)}")
```

```
Training samples:12
Testing samples:3
```

```
from sklearn.linear_model import LinearRegression
```

```
model=LinearRegression()
model.fit(x_train,y_train)
print(f"Slope(coefficient):{model.coef_[0]:.2f}")
print(f"Intercept:{model.intercept_: .2f}")
y_pred=model.predict(x_test)
for actual,predicted in zip(y_test,y_pred):
    print(f"Actual:{actual:>5}|Predicted:{predicted:>7.1f}| "f"Error:{predicted
```

```
Slope(coefficient):5.22
Intercept:27.94
Actual:    82|Predicted:    80.2|Error:-1.8
Actual:    92|Predicted:    90.6|Error:-1.4
Actual:    35|Predicted:    33.2|Error:-1.8
```

```
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
import numpy as np
y_actual=np.array([500,300,700,450,600])
y_pred=np.array([480.320,690,460,590,610])
r2 = r2_score(y_actual, y_pred)
mae = mean_absolute_error(y_actual, y_pred)
mse = mean_squared_error(y_actual, y_pred)
rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
print(f"R2 Score: {r2:.4f} -> model explains {r2 * 100:.1f}% of variance")
print(f"Mean Absolute Error: {mae:.1f} -> avg error +_ {mae:.0f} units")
print(f"Root Mean Squared Error: {rmse:.1f} -> penalized avg error {rmse:.0f
```

```
R2 Score: -1.4977 -> model explains -149.8% of variance
Mean Absolute Error: 159.9 -> avg error +_ 160 units
Root Mean Squared Error: 214.4 -> penalized avg error 214 units
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,mean_squared_error
```

```

np.random.seed(42)
n = 100
data = pd.DataFrame({
    'experience': np.random.randint(0, 20, n), # 0-20 years
    'education': np.random.randint(12, 20, n), # 12-20 years schooling
    'age': np.random.randint(22, 55, n), # 22-55 years
})

```

```

data['salary'] = (
    data['experience'] * 4500 +
    data['education'] * 2000 +
    data['age'] * 300 +
    np.random.normal(0, 3000, n) +
    10000
)

```

```

X = data[['experience', 'education', 'age']]
y = data['salary']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
model = LinearRegression()
model.fit(X_train, y_train)

```

▼ LinearRegression ⓘ ?

LinearRegression()

```

for feat, coef in zip(X.columns, model.coef_):
    print(f" {feat:12s}: ${coef:,.0f} per unit increase")
print(f" Base salary (intercept): ${model.intercept_:,.0f}")
y_pred = model.predict(X_test)
print(f"\nR² : {r2_score(y_test, y_pred):.3f}")
print(f"MAE : ${mean_absolute_error(y_test, y_pred):,.0f}")

```

```

experience   : $4,534 per unit increase
education    : $2,203 per unit increase
age          : $266 per unit increase
Base salary (intercept): $7,694

```

```

R² : 0.988
MAE : $2,219

```

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
np.random.seed(42)
n = 200
locations = ['Urban', 'Suburban', 'Rural']
df = pd.DataFrame({

```

```

'area_sqft': np.random.randint(500, 4000, n),
'num_rooms': np.random.randint(1, 7, n),
'location': np.random.choice(locations, n),
'age_years': np.random.randint(0, 40, n),
})
# Location multipliers
loc_map = {'Urban': 1.5, 'Suburban': 1.0, 'Rural': 0.7}#expensiv medium chea
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 150 +
    df['num_rooms'] * 25000 -
    df['age_years'] * 2000 +
    df['location'].map(loc_map) * 50000 +
    noise
).astype(int)
print(df.head(8).to_string(index=False))
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))

```

```

area_sqft  num_rooms  location  age_years  price
      3674         4    Rural        20  646067
      1360         4    Urban        35  300479
      1794         6    Urban         9  501609
      1630         6    Rural        36  355668
      1595         3    Urban         8  391991
      3592         2    Rural        23  566413
      2138         4  Suburban        34  410974
      2669         1    Urban        34  459191

```

Dataset shape : (200, 5)

```

      area_sqft  num_rooms  age_years  price
count      200.0      200.0      200.0    200.0
mean       2334.0         3.0        20.0  451548.0
std         992.0         2.0        11.0  154871.0
min         521.0         1.0         0.0   86373.0
25%        1519.0         2.0        10.0  326188.0
50%        2372.0         3.0        20.0  458282.0
75%        3230.0         5.0        30.0  575910.0
max         3999.0         6.0        39.0  781261.0

```

```

le = LabelEncoder()
df['location_enc'] = le.fit_transform(df['location'])
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
print(df[['location', 'location_enc']].drop_duplicates().sort_values('locati
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")

```

```

      location  location_enc
0      Rural             0
6  Suburban             1
1      Urban             2

```

Feature matrix shape : (200, 4)
Target vector shape : (200,)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")
```

```
Training samples : 160 (80%)
Testing samples : 40 (20%)
Feature names : ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
```

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
    direction = "↑" if coef > 0 else "↓"
    print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)
```

Learned Coefficients:

```
-----
area_sqft      : Rs      151 ↑
num_rooms      : Rs    23,169 ↑
age_years      : Rs    -2,262 ↓
location_enc   : Rs    18,114 ↑
intercept      : Rs    46,789
-----
```

```
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'✓ Excellent' if r2>0.9 else '✓ Good' if r2>0.7 else '✗ Fair' if r2>0.5 else '✗ Poor'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{'a:>10,'} | {'p:>10,.0f'} | {'p-a:>+10,.0f}'")
```

```
=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9872 ✓ Excellent
MAE : Rs      13,439
RMSE : Rs      16,959
=====
```

Sample Predictions:

Actual	Predicted	Error
489,899	478,320	-11,579
601,329	572,877	-28,452
623,788	650,489	+26,701
291,458	271,086	-20,372
390,040	344,431	-45,609
123,884	134,545	+10,661
484,442	443,295	-41,147
562,823	556,350	-6,473

```
new_houses = pd.DataFrame({
    'area_sqft': [2500, 6800, 940, 7500],
    'num_rooms': [7, 2, 3, 8 ],
    'age_years': [5, 16, 35, 8 ],
    'location_enc': [2, 5, 0, 3 ],
    'location_lbl': ['Urban', 'Suburban', 'Rural', 'Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
    print()
```

HOUSE PRICE PREDICTIONS

```
=====
House 1: 2500 sqft | 7 rooms | 5yr old | Urban
Estimated Price → Rs    610,350

House 2: 6800 sqft | 2 rooms | 16yr old | Suburban
Estimated Price → Rs  1,171,479

House 3: 940 sqft | 3 rooms | 35yr old | Rural
Estimated Price → Rs    178,670

House 4: 7500 sqft | 8 rooms | 8yr old | Urban
Estimated Price → Rs  1,397,771
```

Start coding or [generate](#) with AI.


```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y_reg = np.array([30, 40, 45, 55, 65, 70, 80, 90])
y_cls = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

```
lin = LinearRegression()
lin.fit(X, y_reg)
print('Regression output:', lin.predict([[0.73]]))
```

Regression output: [27.73392857]

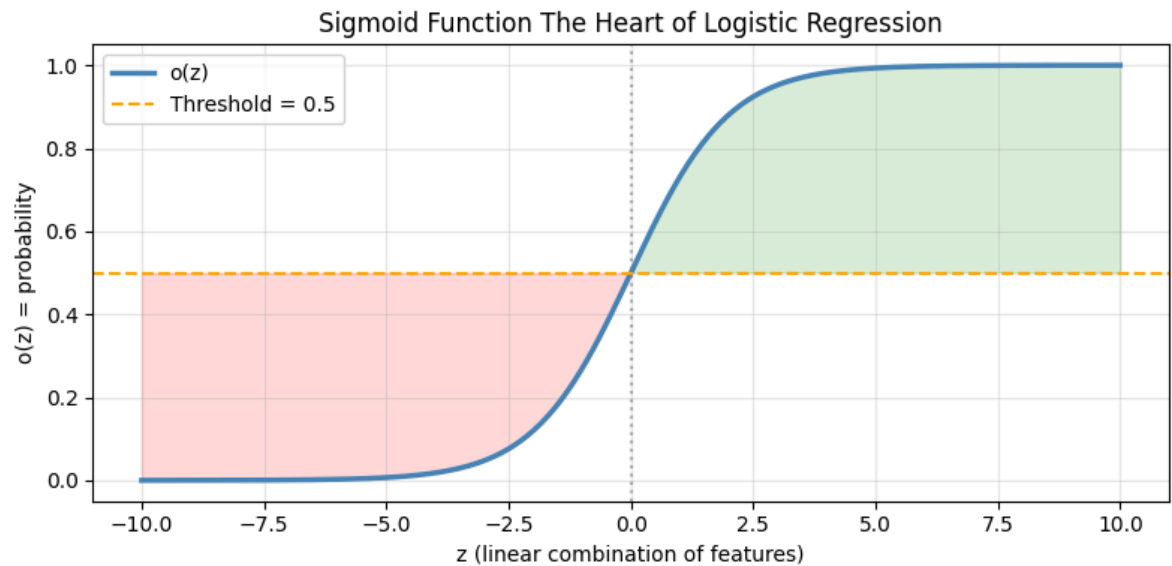
```
log_model = LogisticRegression()
log_model.fit(X, y_cls)
print('Classification prediction:', log_model.predict([[0.73]]))
print('Probability output:', log_model.predict_proba([[0.73]]))
```

Classification prediction: [0]
Probability output: [[0.98799263 0.01200737]]

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 300)
```

```
prob = sigmoid(z)
plt.figure(figsize=(8,4))
plt.plot(z, prob, color='steelblue', linewidth=2.5, label='o(z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)
plt.fill_between(z, prob, 0.5, where=(prob > 0.5), alpha=0.15, color='green')
plt.fill_between(z, prob, 0.5, where=(prob < 0.5), alpha=0.15, color='red')
plt.xlabel('z (linear combination of features)')
plt.ylabel('o(z) = probability')
plt.title('Sigmoid Function The Heart of Logistic Regression')
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
print(f'sigmoid(0) = {sigmoid(0):.4f}')
print(f'sigmoid(5) = {sigmoid(5):.4f}')
print(f'sigmoid(-5) = {sigmoid(-5):.4f}')
```



`sigmoid(0) = 0.5000`

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split

x, y = make_classification(n_samples=500, n_features=5, weights=[0.7, 0.3],
                           x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, ran
model = LogisticRegression()
model.fit(x_train, y_train)
probs = model.predict_proba(x_test)[:, 1]
```

```
print(f'{'Threshold':>10} {'Precision':>10} {'Recall':>8} {'F1':>6}')
print('-'*45)
for threshold in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    preds = (probs >= threshold).astype(int)
    p = precision_score(y_test, preds, zero_division=0) #from
    r = recall_score(y_test, preds) # from all positive to got
    f = f1_score(y_test, preds, zero_division=0) # balnce btw
    print(f'{'threshold':>10.1f} {'p':>10.3f} {'r':>8.3f} {'f':>6.3f}')
```

Threshold	Precision	Recall	F1

0.2	0.696	0.941	0.800
0.3	0.762	0.941	0.842
0.4	0.882	0.882	0.882
0.5	0.882	0.882	0.882
0.6	0.903	0.824	0.862
0.7	0.903	0.824	0.862
0.8	0.958	0.676	0.793

```
import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

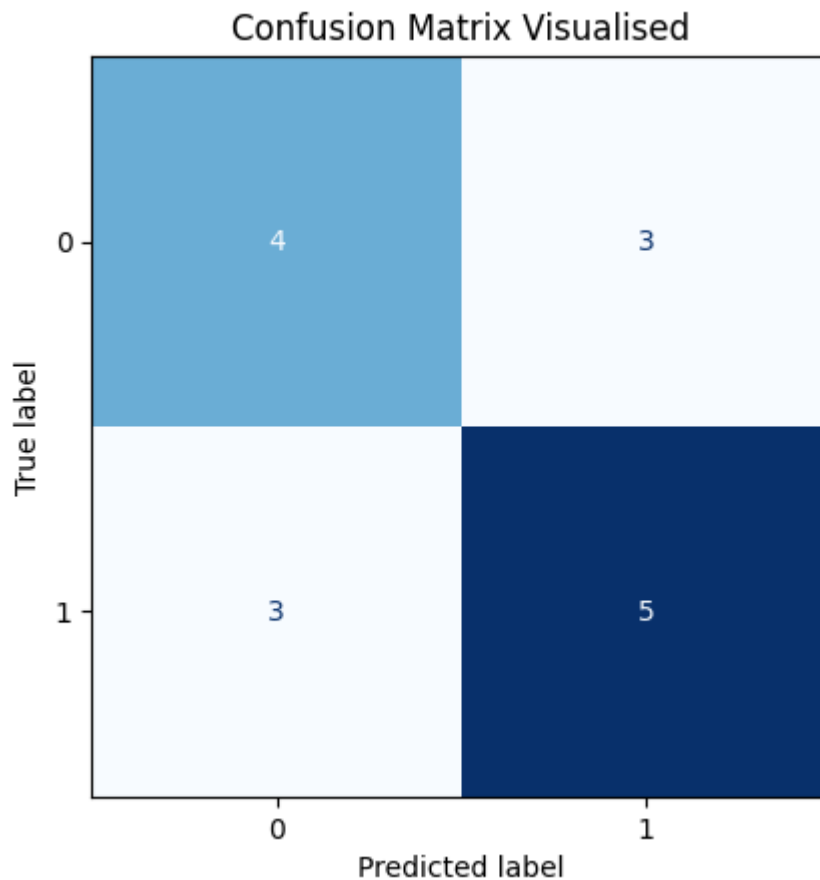
```
y_true = [1,1,1,1,1, 0,0,0,0,0, 1,0,1,0,1]
y_pred = [1,1,1,0,0, 0,0,0,1,1, 1,0,0,1,1]
y_true = np.array(y_true)
y_pred = np.array(y_pred)
TP = ((y_pred == 1) & (y_true == 1)).sum()
TN = ((y_pred == 0) & (y_true == 0)).sum()
FP = ((y_pred == 1) & (y_true == 0)).sum()
FN = ((y_pred == 0) & (y_true == 1)).sum()

print(f'TP = {TP}, TN={TN}, FP={FP}, FN={FN}')
print(f'Accuracy = (TP+TN)/Total = {(TP+TN)/len(y_true)} = {(TP+TN)/len(y_true):.2f}')
print(f'Precision = TP/(TP+FP) = {TP}/{(TP)+{FP}} = {TP/(TP+FP):.2f}')
print(f'Recall = TP/(TP+FN) = {TP}/{(TP)+{FN}} = {TP/(TP+FN):.2f}')

cm=confusion_matrix(y_true, y_pred)
print('confusion matrix:\n', cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', colorbar=False)
plt.title('Confusion Matrix Visualised'); plt.show()
```

TP = 5, TN=4, FP=3, FN=3
 Accuracy = (TP+TN)/Total = 0.6 = 0.60
 Precision = TP/(TP+FP) = 5/(5+3) = 0.62
 Recall = TP/(TP+FN) = 5/(5+3) = 0.62
 confusion matrix:
 [[4 3]
 [3 5]]



```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score, accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np

```

```

X_bal, y_bal = make_classification(n_samples=1000, weights=[0.5, 0.5], random_state=42)
x_imb, y_imb = make_classification(n_samples=1000, weights=[0.9, 0.1], random_state=42)

```

```

def evaluate(x, y, label):
    X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
    model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    print(f'\n=== {label} ===')
    print(f'Accuracy: {acc:.2%} <- ({"MISLEADING!" if f1 < 0.5 else "OK"})')
    print(f'F1 Score: {f1:.2%} <- Real performance')
    print(classification_report(y_test, y_pred, zero_division=0))

```

```
evaluate(X_bal, y_bal, "Balanced Data")
evaluate(x_imb, y_imb, "Imbalanced Data")
```

```
=== Balanced Data ===
```

```
Accuracy: 80.50% <- (OK)
```

```
F1 Score: 80.00% <- Real performance
```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```
=== Imbalanced Data ===
```

```
Accuracy: 90.00% <- (MISLEADING!)
```

```
F1 Score: 37.50% <- Real performance
```

	precision	recall	f1-score	support
0	0.92	0.97	0.95	179
1	0.55	0.29	0.38	21
accuracy			0.90	200
macro avg	0.73	0.63	0.66	200
weighted avg	0.88	0.90	0.89	200

```
X_train, X_test, y_train, y_test = train_test_split(X_bal, y_bal, test_size=
model_fixed = LogisticRegression(class_weight='balanced', max_iter=1000)
model_fixed.fit(X_train, y_train)
y_pred_fixed = model_fixed.predict(X_test)
print('\n=== After class_weight=balanced ===')
print(classification_report(y_test, y_pred_fixed, zero_division=0))
```

```
=== After class_weight=balanced ===
```

	precision	recall	f1-score	support
0	0.84	0.80	0.82	106
1	0.79	0.83	0.81	94
accuracy			0.81	200
macro avg	0.81	0.82	0.81	200
weighted avg	0.82	0.81	0.82	200

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
X, y = make_classification(n_samples=800, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
scalar=StandardScaler()
X_train=scalar.fit_transform(X_train)
X_test=scalar.transform(X_test)
```

```
lr = LogisticRegression().fit(X_train, y_train)
prob_lr = lr.predict_proba(X_test)[:, 1]
```

```
dt = DecisionTreeClassifier(max_depth=4).fit(X_train, y_train)
prob_dt = dt.predict_proba(X_test)[:, 1]
```

```
dt = DecisionTreeClassifier(max_depth=4).fit(X_train, y_train)
prob_dt = dt.predict_proba(X_test)[:, 1]
```

```
fpr_lr, tpr_lr, _ = roc_curve(y_test, prob_lr)
fpr_dt, tpr_dt, _ = roc_curve(y_test, prob_dt)
auc_lr = roc_auc_score(y_test, prob_lr)
auc_dt = roc_auc_score(y_test, prob_dt)
```

```
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=2.5, color='steelblue', label=f'Logistic Reg AUC')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve – Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

ROC Curve — Model Comparison

```
print(f'Logistic Regression AUC: {auc_lr:.4f}')
print(f'Decision Tree AUC: {auc_dt:.4f}')
print('Higher AUC = better model at separating classes')
```

```
Logistic Regression AUC: 0.9833
Decision Tree AUC: 0.9761
Higher AUC = better model at separating classes
```

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt,
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion
ConfusionMatrixDisplay, roc_auc_score, roc_curve)
```

```
df = pd.read_csv('https://raw.githubusercontent.com/plotly/d
print(df.shape) # (768, 9)
print(df['Outcome'].value_counts()) # 500 healthy, 268 diabet
print(df.describe().round(2)) # stats: mean/std/min/max
# Distribution of each feature vs outcome
df.hist(bins=25, figsize=(14,10), color='steelblue', edgecolor
plt.suptitle('Feature Distributions', fontsize=14); plt.tight
```

```
(768, 9)
```

```
Outcome
```

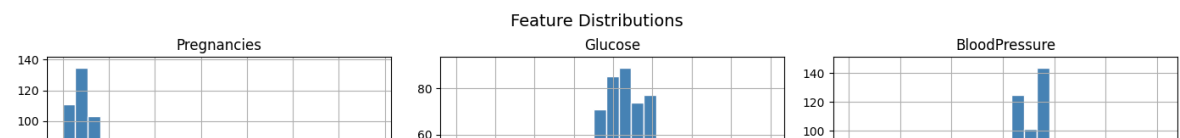
```
0    500
```

```
1    268
```

```
Name: count, dtype: int64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
count	768.00	768.00	768.00	768.00	768.00	768.00
mean	3.85	120.89	69.11	20.54	79.80	31.99
std	3.37	31.97	19.36	15.95	115.24	7.88
min	0.00	0.00	0.00	0.00	0.00	0.00
25%	1.00	99.00	62.00	0.00	0.00	27.30
50%	3.00	117.00	72.00	23.00	30.50	32.00
75%	6.00	140.25	80.00	32.00	127.25	36.60
max	17.00	199.00	122.00	99.00	846.00	67.10

	DiabetesPedigreeFunction	Age	Outcome
count	768.00	768.00	768.00
mean	0.47	33.24	0.35
std	0.33	11.76	0.48
min	0.08	21.00	0.00
25%	0.24	24.00	0.00
50%	0.37	29.00	0.00
75%	0.63	41.00	1.00
max	2.42	81.00	1.00





```
In [ ]: import numpy as np
        from sklearn.tree import DecisionTreeClassifier
        import matplotlib.pyplot as plt
```

```
In [ ]: def gini_impurity(labels):
        """Calculate Gini impurity for a list of class labels."""
        labels = np.array(labels)
        n = len(labels)
        if n == 0: return 0
        classes = np.unique(labels)
        gini = 1.0
        for c in classes:
            p = np.sum(labels == c) / n # proportion of class c
            gini -= p ** 2
        return gini
```

```
In [ ]: print('Pure node (all same) :', gini_impurity([1,1,1,1,1])) # 0.00
        print('50-50 split :', gini_impurity([0,0,0,1,1,1])) # 0.50
        print('70-30 split :', gini_impurity([0,0,0,0,0,0,0,1,1,1])) # 0.42
        print('90-10 split :', gini_impurity([0]*9 + [1])) # 0.18
```

```
Pure node (all same) : 0.0
50-50 split : 0.5
70-30 split : 0.42000000000000004
90-10 split : 0.17999999999999994
```

```
In [ ]: def weighted_gini(left_labels, right_labels):
        """Gini after a split – weighted by group size."""
        n_total = len(left_labels) + len(right_labels)
        w_left = len(left_labels) / n_total
        w_right = len(right_labels) / n_total
        return w_left * gini_impurity(left_labels) + w_right * gini_impurity(right_labels)
```

```
In [ ]: left_A = [0,0,0,0,1]
        right_A = [0,1,1,1,1]
```

```
In [ ]: left_B = [0,0,1,1,0]
        right_B = [0,1,0,1,1]
```

```
In [ ]: print(f'Weighted Gini – Glucose split: {weighted_gini(left_A, right_A):.3f}')
        print(f'Weighted Gini – Age split : {weighted_gini(left_B, right_B):.3f}')
        print('Tree picks the split with LOWER weighted Gini → Glucose split is better')
```

```
Weighted Gini – Glucose split: 0.320
Weighted Gini – Age split : 0.480
Tree picks the split with LOWER weighted Gini → Glucose split is better!
```

```
In [ ]: from sklearn.datasets import make_classification
        from sklearn.tree import export_text
        X, y = make_classification(n_samples=200, n_features=4, random_state=42)
        tree = DecisionTreeClassifier(max_depth=2, random_state=42)
        tree.fit(X, y)
        print(export_text(tree, feature_names=['F1', 'F2', 'F3', 'F4']))
```

```

| --- F1 <= -0.15
|   | --- F3 <= -0.44
|   |   | --- class: 0
|   |   | --- F3 > -0.44
|   |   | --- class: 0
| --- F1 > -0.15
|   | --- F3 <= 0.63
|   |   | --- class: 1
|   |   | --- F3 > 0.63
|   |   | --- class: 0

```

```

In [ ]: from sklearn.tree import DecisionTreeClassifier
        from sklearn.datasets import make_classification
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score
        import matplotlib.pyplot as plt

```

```

In [ ]: x,y=make_classification(n_samples=200,n_features=7,random_state=42)
        x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=
        depths=range(1,11)
        train_acc,test_acc=[], []

```

```

In [ ]: train_acc, test_acc = [], []
        for d in depths:
            tree=DecisionTreeClassifier(max_depth=d,random_state=42)
            tree.fit(x_train,y_train)
            train_acc.append(accuracy_score(y_train,tree.predict(x_train)))
            test_acc.append(accuracy_score(y_test,tree.predict(x_test)))

```

```

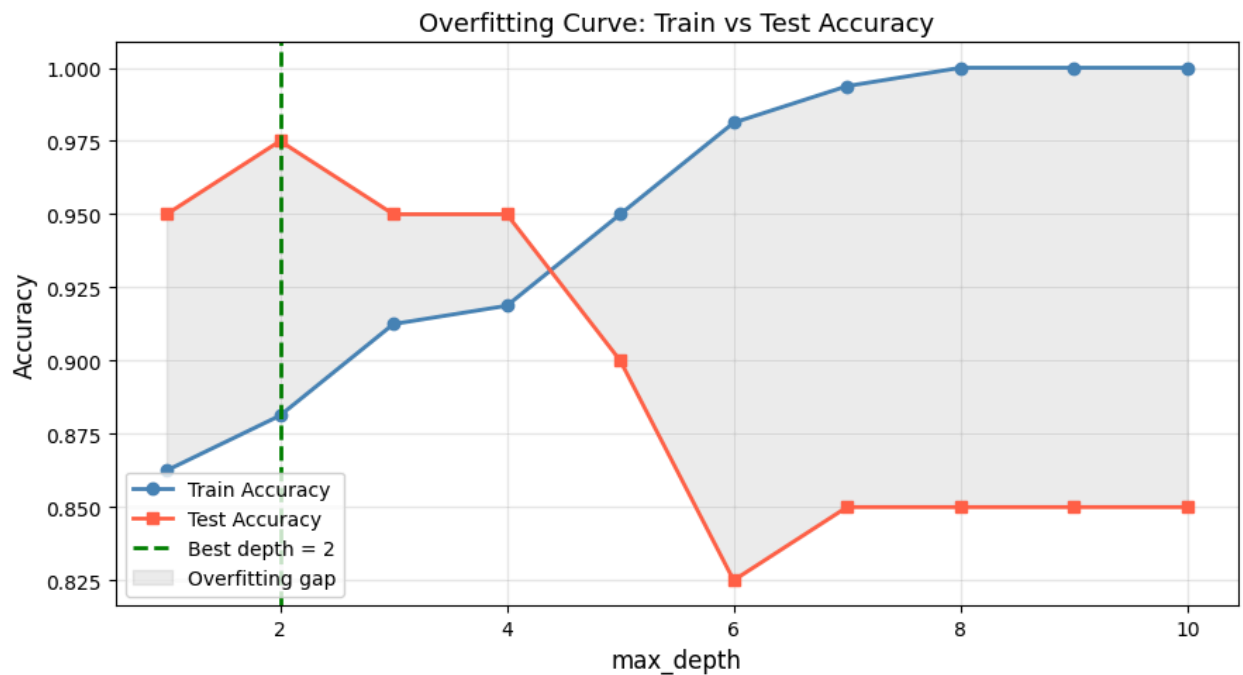
In [ ]: best_depth=depths[test_acc.index(max(test_acc))]

```

```

In [ ]: plt.figure(figsize=(9, 5))
        plt.plot(depths, train_acc, 'o-', color='steelblue', lw=2, label='Train Accuracy')
        plt.plot(depths, test_acc, 's-', color='tomato', lw=2, label='Test Accuracy')
        plt.axvline(x=best_depth, color='green', linestyle='--', lw=2,
                    label=f'Best depth = {best_depth}')
        plt.fill_between(depths,
                        train_acc, test_acc,
                        alpha=0.15, color='gray', label='Overfitting gap')
        plt.xlabel('max_depth', fontsize=12)
        plt.ylabel('Accuracy', fontsize=12)
        plt.title('Overfitting Curve: Train vs Test Accuracy', fontsize=13)
        plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
        print(f'Best test accuracy {max(test_acc):.2%} at depth={best_depth}')

```



Best test accuracy 97.50% at depth=2

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
        from sklearn.model_selection import cross_val_score
        import numpy as np
```

```
In [ ]: configs=[
        {'max_depth':None},
        {'max_depth':5},
        {'max_depth':5,'min_samples_split':10},
        {'max_depth':5,'min_samples_leaf':5},
        {'max_depth':5,'max_features':'sqrt'},
    ]
```

```
In [ ]: print(f'Config:45s {CV Score(mean+/-std):20s}')
        print('-'*60)
```

Config	CV Score(mean+/-std)

```
In [ ]: for cfg in configs:
        tree=DecisionTreeClassifier(**cfg,random_state=42)
        scores=cross_val_score(tree,x,y,cv=5,scoring='accuracy')
        print(f'{str(cfg):45s} {scores.mean():.3f} +/- {scores.std():.3f}')
```

{'max_depth': None}	0.795 +/- 0.062
{'max_depth': 5}	0.810 +/- 0.044
{'max_depth': 5, 'min_samples_split': 10}	0.795 +/- 0.048
{'max_depth': 5, 'min_samples_leaf': 5}	0.820 +/- 0.040
{'max_depth': 5, 'max_features': 'sqrt'}	0.660 +/- 0.060



```
In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_diabetes # regression dataset repurposed as
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer
import pandas as pd, numpy as np, matplotlib.pyplot as plt
```

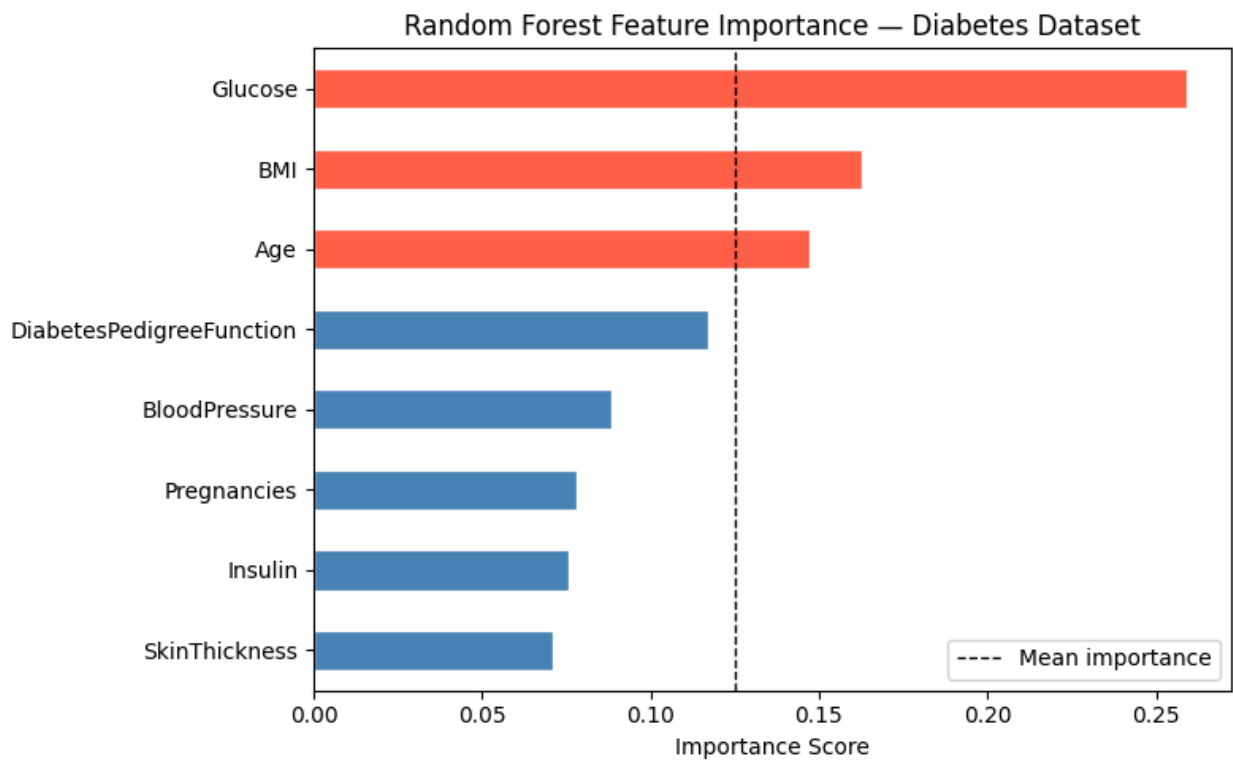
```
In [ ]: import pandas as pd
import os
if not os.path.exists('diabetes.csv'):
    !wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv
df = pd.read_csv('diabetes.csv')
fix_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in fix_cols: df[col] = df[col].replace(0, df[col].median())
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
```

```
Out[ ]: RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)
```

```
In [ ]: imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print('Feature Importances (higher = more important):')
print(imp.round(3))
```

```
Feature Importances (higher = more important):
Glucose      0.259
BMI           0.163
Age           0.147
DiabetesPedigreeFunction  0.117
BloodPressure  0.088
Pregnancies   0.078
Insulin       0.076
SkinThickness 0.071
dtype: float64
```

```
In [ ]: imp_sorted = imp.sort_values()
colors = ['tomato' if v > imp.mean() else 'steelblue' for v in imp_sorted]
imp_sorted.plot(kind='barh', color=colors, figsize=(8,5), edgecolor='white')
plt.axvline(x=imp.mean(), color='black', linestyle='--', linewidth=1, label='Mean')
plt.title('Random Forest Feature Importance - Diabetes Dataset', fontsize=12)
plt.xlabel('Importance Score'); plt.legend(); plt.tight_layout(); plt.show()
```



```
In [ ]: top_features = imp[imp > imp.mean()].index.tolist()
print(f'\nTop features (above average): {top_features}')
rf_top = RandomForestClassifier(n_estimators=200, random_state=42)
rf_top.fit(X_train[top_features], y_train)
full_acc = rf.score(X_test, y_test)
top_acc = rf_top.score(X_test[top_features], y_test)
print(f'Accuracy with all {len(X.columns)} features : {full_acc:.2%}')
print(f'Accuracy with top {len(top_features)} features : {top_acc:.2%}')
print('Often comparable – low-importance features can add noise!')
```

Top features (above average): ['Glucose', 'BMI', 'Age']
 Accuracy with all 8 features : 77.27%
 Accuracy with top 3 features : 72.73%
 Often comparable – low-importance features can add noise!

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np, matplotlib.pyplot as plt, time

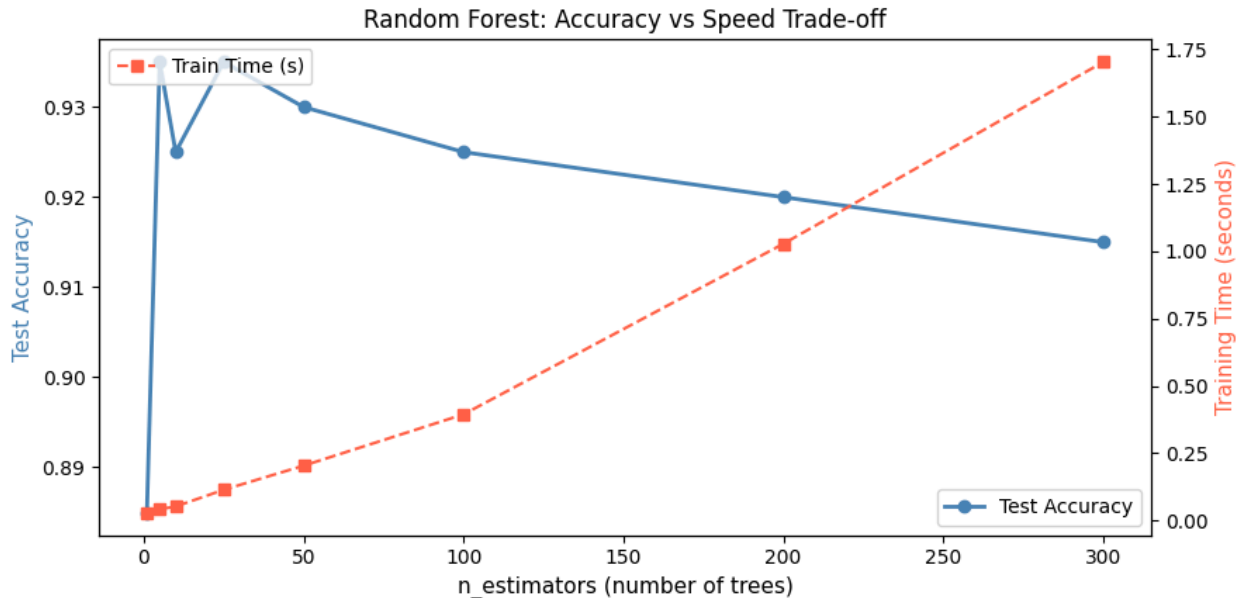
X, y = make_classification(n_samples=1000, n_features=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [ ]: rf_oob = RandomForestClassifier(n_estimators=10, oob_score=True, random_state=42)
rf_oob.fit(X_train, y_train)
print(f'OOB Score : {rf_oob.oob_score:.2%} ← estimated on ~37% unseen rows per tree')
print(f'Test Score : {rf_oob.score(X_test, y_test):.2%} ← actual held-out test score')
print('OOB score is a free estimate – no need for a separate validation set!')
```

OOB Score : 85.25% ← estimated on ~37% unseen rows per tree
Test Score : 92.50% ← actual held-out test
OOB score is a free estimate – no need for a separate validation set!

```
/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_forest.py:612: UserWarning: Some inputs do not have OOB scores. This probably means too few trees were used to compute any reliable OOB estimates.  
warn(
```

```
In [ ]: n_trees_list = [1, 5, 10, 25, 50, 100, 200, 300]  
accs, times = [], []  
for n in n_trees_list:  
    t0 = time.time()  
    rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=-1)  
    rf.fit(X_train, y_train)  
    accs.append(accuracy_score(y_test, rf.predict(X_test)))  
    times.append(time.time() - t0)  
fig, ax1 = plt.subplots(figsize=(9, 4.5))  
ax2 = ax1.twinx()  
ax1.plot(n_trees_list, accs, 'o-', color='steelblue', lw=2, label='Test Accuracy')  
ax2.plot(n_trees_list, times, 's--', color='tomato', lw=1.5, label='Train Time (s)')  
ax1.set_xlabel('n_estimators (number of trees)', fontsize=11)  
ax1.set_ylabel('Test Accuracy', color='steelblue', fontsize=11)  
ax2.set_ylabel('Training Time (seconds)', color='tomato', fontsize=11)  
ax1.set_title('Random Forest: Accuracy vs Speed Trade-off')  
ax1.legend(loc='lower right'); ax2.legend(loc='upper left')  
plt.tight_layout(); plt.show()
```



In []:

Model Evaluation & Choosing the Right Model

Why Evaluation Matters

Training a model is easy. Knowing if it actually works is the hard part. A model that scores 98% on training data might score 62% on new data. This gap is the real test of whether your model has learned a general pattern or just memorised the training examples.

Building a Machine Learning model is only half the job.

The real question is:

"Will the model work well on new, unseen data?"

A model may achieve:

98% Training Accuracy 62% Test Accuracy

This means the model has memorized the training data instead of learning general patterns.

Therefore, evaluating a model correctly is essential before deploying it in the real world.

Types of Error

Error Type	What It Measures	What It Tells You
Training Error	Error on training data	Low error means the model learned the training data (or mer
Validation Error	Error on validation data	Helps detect overfitting and tune the model.
Test Error	Error on completely unseen data	Best estimate of real-world performance.

Concept	Key Idea
Training Error	Measures performance on seen data
Validation Error	Helps tune and improve models
Test Error	Estimates real-world performance
Overfitting	Excellent training performance but poor test performance
Golden Rule	Never repeatedly use the test set for model tuning

The Golden Rule

NEVER use your test set more than once. Every time you peek at test results and adjust your model, you're leaking information. The final test score must come from data the model has NEVER influenced in any way

See Training Error vs Test Error Gap Watch the gap grow as you increase model complexity:

```
# ■ Training Error vs Test Error Gap ■
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt, numpy as np

X, y = make_classification(
    n_samples=500,
    n_features=10,
    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42)

train_errs, test_errs = [], []

depths = range(1, 25)

for d in depths:

    tree = DecisionTreeClassifier(
        max_depth=d,
        random_state=42)

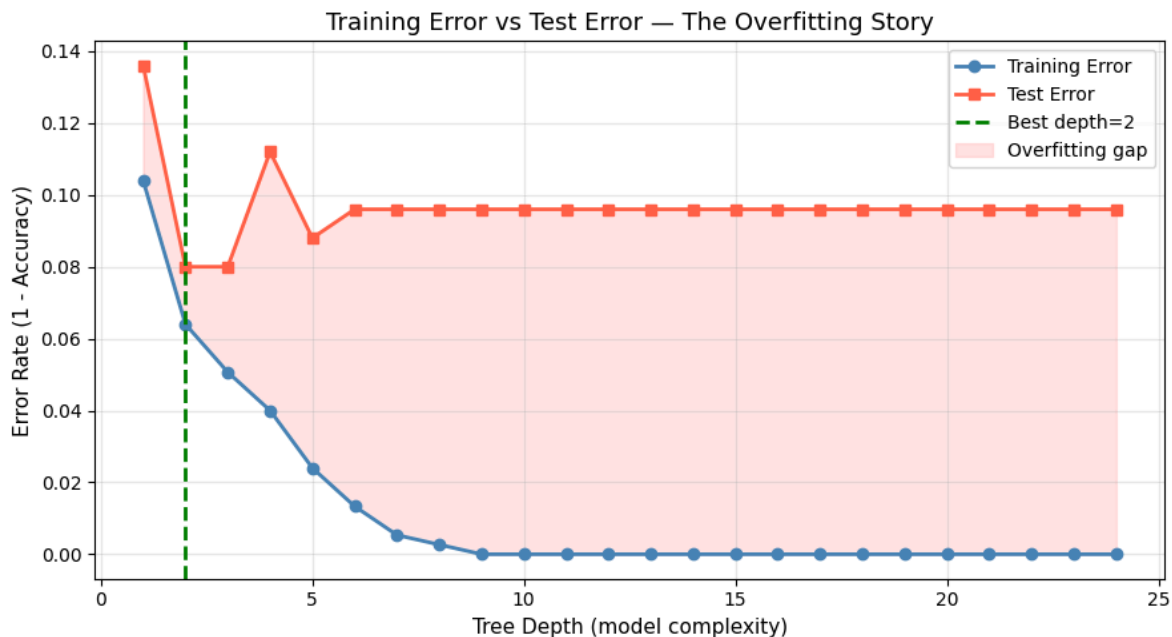
    tree.fit(X_train, y_train)

    train_errs.append(1 - accuracy_score(y_train, tree.predict(X_train)))
    test_errs.append(1 - accuracy_score(y_test, tree.predict(X_test)))

best = depths[test_errs.index(min(test_errs))]

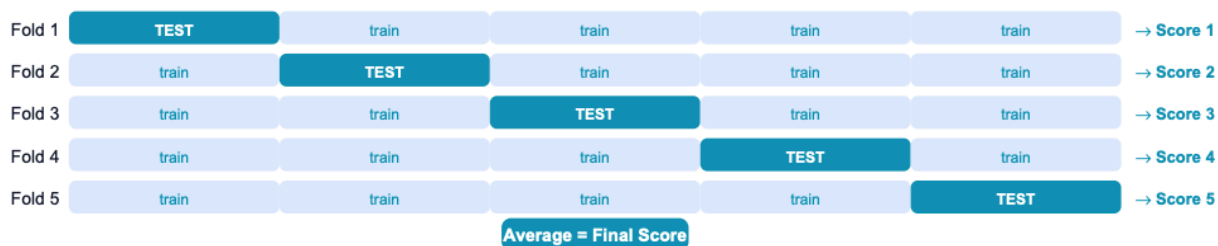
plt.figure(figsize=(9, 5))
plt.plot(depths, train_errs, 'o-', lw=2, color='steelblue', label='Training')
plt.plot(depths, test_errs, 's-', lw=2, color='tomato', label='Test Error')
plt.axvline(x=best, color='green', lw=2, linestyle='--', label=f'Best depth=')
plt.fill_between(depths, train_errs, test_errs, alpha=0.1, color='red', label='')
plt.xlabel('Tree Depth (model complexity)', fontsize=11)
```

```
plt.ylabel('Error Rate (1 - Accuracy)', fontsize=11)
plt.title('Training Error vs Test Error – The Overfitting Story', fontsize=11)
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
# STORY: As depth grows, train error → 0 (perfect memorisation)
# But test error has a minimum – that's the sweet spot
```



Cross-Validation

A single train/test split gives you ONE score — but what if that split was unlucky? Maybe all the hard examples landed in your test set. K-Fold Cross-Validation solves this by using K different splits and averaging the scores.



Each fold is used as the test set exactly once. The final score is the mean of all 5 fold scores.

K Value	Trade-Off	Recommended Use
K = 5	Fast and reliable	Default choice; works well for most datasets
K = 10	More reliable but slower	Preferred for smaller datasets (< 1000 rows)
K = 3	Very fast but slightly less stable	Large datasets (> 100k rows) where 5-fold CV be
K = N (Leave-One-Out CV)	Most reliable but extremely slow	Very small datasets (< 100 rows) where every sam

K-Fold Cross Validation Idea

Instead of testing only once:

Divide data into K equal parts (folds)

Use one fold as the test set

Use remaining folds as training data

Repeat until every fold has been used as a test set

Average all scores

Cross-Validation Three Ways

From the simplest one-liner to full manual control:

```
# ■■■ Cross-Validation: One-liner + StratifiedKFold + Pipeline ■■■
from sklearn.datasets import make_classification
from sklearn.model_selection import (cross_val_score, KFold,
StratifiedKFold, cross_validate)
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np
```

```
X, y = make_classification(n_samples=600, n_features=10, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
# ■■■ Method 1: Simplest one-liner ■■■
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'5-Fold CV Scores: {scores.round(3)}')
print(f'Mean: {scores.mean():.3f} Std: {scores.std():.3f}')
print(f'Reliable range: {scores.mean()-2*scores.std():.3f} - {scores.mean()+
```

```
2*scores.std():.3f}')
5-Fold CV Scores: [0.858 0.933 0.908 0.9 0.942]
Mean: 0.908 Std: 0.029
Reliable range: 0.850 - 0.967
```

```
# ■■■ Method 2: StratifiedKFold (keeps class ratio in each fold) ■■■
# Use this for classification, especially with imbalanced classes
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
strat_scores = cross_val_score(model, X, y, cv=skf, scoring='f1_macro')
print(f'\nStratified 5-Fold F1: {strat_scores.mean():.3f} ± {strat_scores.st
```

```
deviation():.3f}')
Stratified 5-Fold F1: 0.908 ± 0.027
```

```
# ■■■ Method 3: cross_validate – get multiple metrics at once ■■■
results = cross_validate(model, X, y, cv=5,
scoring=['accuracy', 'f1_macro', 'roc_auc'],
return_train_score=True)
print('\ncross_validate results:')
for metric in ['accuracy', 'f1_macro', 'roc_auc']:
```

```
train_k = f'train_{metric}'
test_k = f'test_{metric}'
print(f' {metric:12s}: train={results[train_k].mean():.3f} test={results[t
```

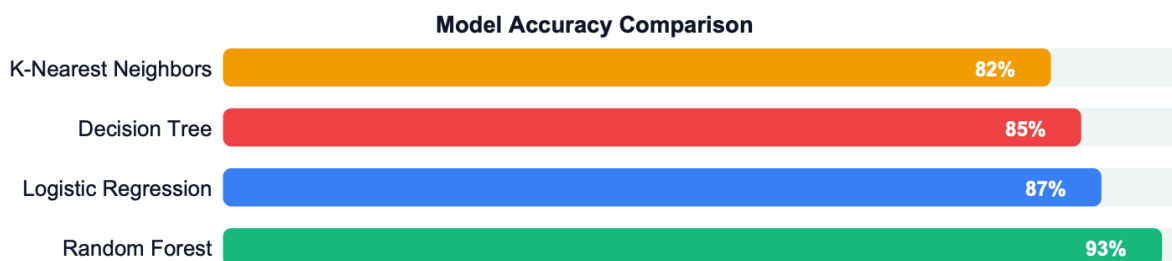
```
cross_validate results:
accuracy      : train=1.000 test=0.908
f1_macro      : train=1.000 test=0.908
roc_auc       : train=1.000 test=0.970
```

```
# ■ Method 4: Pipeline + CV (scale inside CV – no data leakage!) ■
pipe = Pipeline([
('scaler', StandardScaler()), # Scale inside each fold
('model', LogisticRegression(max_iter=1000))
])
pipe_scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print(f'\nPipeline CV (no leakage): {pipe_scores.mean():.3f} ± {pipe_scores.
```

Pipeline CV (no leakage): 0.915 ± 0.030

Comparing Multiple Models

There is no single best algorithm for every problem. The right approach is to train several models, evaluate them on the same data with the same metric, and choose based on performance, speed, and interpretability needs.



Random Forest wins on accuracy, but Decision Tree is fastest and most interpretable.

Model	Best For	
Logistic Regression	Linearly separable data, binary classification, probability prediction	Data contain
Decision Tree	Need easy interpretation, rule-based decisions, mixed feature types	Large data
Random Forest	General-purpose classification, high accuracy, robust predictions	Very large c
K-Nearest Neighbors (KNN)	Small datasets, similarity-based problems, simple classification tasks	High-dimer

Build a Model Leaderboard with One Loop

Train 5 models on the same data, collect metrics, build a comparison DataFrame:

```
# ■ Model Comparison Leaderboard ■
from sklearn.datasets import make_classification
from sklearn.model_selection import StratifiedKFold, cross_validate
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import pandas as pd, numpy as np, matplotlib.pyplot as plt
X, y = make_classification(n_samples=800, n_features=15, random_state=42)

```

```

# ■ Define models (wrapped in Pipeline to include scaling) ■
models = {
    'Logistic Regression' : Pipeline([('sc',StandardScaler()),('m',LogisticRegre
    'Decision Tree' : DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest' : RandomForestClassifier(n_estimators=100, random_state=42),
    'KNN (k=5)' : Pipeline([('sc',StandardScaler()),('m',KNeighborsClassifier(n_
    })
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = []

for name, model in models.items():

    cv_res = cross_validate(
        model,
        X,
        y,
        cv=cv,
        scoring=['accuracy', 'f1_macro', 'roc_auc'],
        return_train_score=False
    )

    results.append({
        'Model': name,
        'Accuracy': cv_res['test_accuracy'].mean(),
        'F1 Score': cv_res['test_f1_macro'].mean(),
        'AUC-ROC': cv_res['test_roc_auc'].mean(),
        'CV Std': cv_res['test_accuracy'].std(),
    })

print(
    f'{name:22s}: '
    f'Acc={results[-1]["Accuracy"]:.3f} '
    f'F1={results[-1]["F1 Score"]:.3f} '
    f'AUC={results[-1]["AUC-ROC"]:.3f} '
)

```

```

Logistic Regression : Acc=0.930 F1=0.930 AUC=0.975
Decision Tree       : Acc=0.925 F1=0.925 AUC=0.948
Random Forest      : Acc=0.936 F1=0.936 AUC=0.977
KNN (k=5)          : Acc=0.886 F1=0.886 AUC=0.949

```

```
# ■■■ Build the leaderboard ■■■
df_res = pd.DataFrame(results).sort_values('Accuracy', ascending=False)
df_res[['Accuracy', 'F1 Score', 'AUC-ROC', 'CV Std']] = df_res[['Accuracy', 'F1
print('\n' + '='*70)
print('MODEL LEADERBOARD')
print('='*70)
print(df_res.to_string(index=False))
```

```
=====
MODEL LEADERBOARD
=====
```

	Model	Accuracy	F1 Score	AUC-ROC	CV Std
	Random Forest	0.9362	0.9362	0.9775	0.0310
	Logistic Regression	0.9300	0.9300	0.9752	0.0222
	Decision Tree	0.9250	0.9249	0.9481	0.0265
	KNN (k=5)	0.8863	0.8860	0.9487	0.0242

Hyperparameter Tuning — GridSearchCV

Hyperparameters are settings you choose before training starts — things like `max_depth`, `n_estimators`, `learning_rate`. The model cannot learn these from data; you must set them. GridSearchCV automates the search by trying every combination.

Parameters vs Hyperparameters

Feature	Parameters
What are they?	Values learned automatically from the training data
Who decides them?	The machine learning algorithm learns them
Examples	Weights in Logistic Regression, Split Thresholds in Decision Trees, Coefficients
How are they set?	Automatically learned during training
Changed During Training?	Yes, continuously updated while learning
Purpose	Help the model learn patterns from data
Can We Directly Control Them?	Usually No

GridSearchCV — All Combinations

	n=50	n=100	n=200
d=2	0.78	0.82	0.80
d=4	0.84	0.89	0.87
d=6	0.85	0.91 ★ BEST	0.90
d=8	0.84	0.90	0.89

How GridSearchCV Works

1. You give it a grid of values to try.
2. It trains the model for every combination.
3. Each combination is evaluated with K-fold CV.
4. It reports the combination with the best mean CV score.

The table shows all depth × `n_estimators` combos. **Dark blue = best combo.**

GridSearchCV + RandomizedSearchCV

Grid search tries everything; random search is faster for large grids:

```
# ■■ GridSearchCV
from sklearn.model_selection import GridSearchCV StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
import pandas as pd, numpy as np
X, y = make_classification(n_samples=600, n_features=12, random_state=42)
```

```
# ■■ Part 1: GridSearchCV for Decision Tree ■■
# Hyperparameters to test
param_grid = {
    'max_depth': [2, 4, 6, 8, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 3, 5],
    'criterion': ['gini', 'entropy']
}

# 5-Fold Stratified Cross Validation
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

# Grid Search
grid_search = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
)

# Train and find best combination
grid_search.fit(X, y)

# Results
print("Best Parameters:", grid_search.best_params_)
print("Best Accuracy:", round(grid_search.best_score_, 4))
```

```
Best Parameters: {'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 1}
Best Accuracy: 0.9083
```

```
# Show GridSearch results
results = pd.DataFrame(grid_search.cv_results_)

# Top 5 models
top_5 = results[
    ['param_max_depth', 'param_criterion',
     'mean_test_score', 'rank_test_score']
].sort_values('rank_test_score').head(5)
```

```
print("\nTop 5 Models:")
print(top_5)
```

```
Top 5 Models:
  param_max_depth param_criterion mean_test_score rank_test_score
61                4          entropy          0.908333             1
62                4          entropy          0.908333             1
60                4          entropy          0.908333             1
0                  2             gini          0.903333             4
4                  2             gini          0.903333             4
```

LAB : MODEL COMPARISION DASHBOARD

Build a complete model selection pipeline: load data, preprocess, train 4 models, evaluate with cross-validation, tune hyperparameters, and produce a final comparison report. We use the Pima Indians Diabetes dataset as the benchmark.

Step 1 — Load, Clean, Split

```
# ■ Load and Clean ■
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import StratifiedKFold, cross_validate, GridSea
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, Confusi
import time, warnings; warnings.filterwarnings('ignore')

# Check if 'diabetes.csv' exists, if not, download it
import os
if not os.path.exists('diabetes.csv'):
    # Using wget to download the file from a common public repository
    !wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.

df = pd.read_csv('diabetes.csv')
for col in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']:
    df[col] = df[col].replace(0, df[col].median())
X = df.drop('Outcome', axis=1)
y = df['Outcome']
print(f'Dataset: {X.shape[0]} rows, {X.shape[1]} features')
print(f'Class balance: {y.value_counts().to_dict()}')
# Output: {0: 500, 1: 268} – slightly imbalanced
```

```
--2026-06-05 01:28:22-- https://raw.githubusercontent.com/plotly/datasets/ma
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.10
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.1
HTTP request sent, awaiting response... 200 OK
Length: 23873 (23K) [text/plain]
```

Saving to: 'diabetes.csv'

diabetes.csv 100%[=====>] 23.31K --.-KB/s in 0s

2026-06-05 01:28:22 (81.2 MB/s) - 'diabetes.csv' saved [23873/23873]

Dataset: 768 rows, 8 features

Class balance: {0: 500, 1: 268}

Step 2 — Build Models & Run Cross-Validation

```
# ■ Cross-Validation Leaderboard ■
# Models – LR and KNN need scaling; trees don't (Pipeline handles this clean
models = {
    'Logistic Regression': Pipeline([('sc',StandardScaler()),
    ('m',LogisticRegression(max_iter=1000,random_state=42))]),
    'Decision Tree' : DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest' : RandomForestClassifier(n_estimators=100, random_state=42),
    'KNN' : Pipeline([('sc',StandardScaler()),
    ('m',KNeighborsClassifier(n_neighbors=7))]),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
metrics = ['accuracy','f1_macro','roc_auc','precision_macro','recall_macro']
records = []
for name, model in models.items():
    t0 = time.time()
    res = cross_validate(model, X, y, cv=cv, scoring=metrics)
    elapsed = time.time()-t0
    row = {'Model': name, 'Time(s)': round(elapsed,2)}
    for m in metrics:
        row[m.replace('_macro','').title()] = round(res[f'test_{m}'].mean(), 4)
        row[f'{m.replace('_macro','').title()}_std'] = round(res[f'test_{m}'].std(), 4)
    records.append(row)
leaderboard = pd.DataFrame(records).sort_values('Accuracy', ascending=False)
print(leaderboard[['Model','Accuracy','F1','Roc_Auc','Precision','Recall','Time(s)']])
```

Model	Accuracy	F1	Roc_Auc	Precision	Recall	Time(s)
Logistic Regression	0.7708	0.7320	0.8372	0.7602	0.7219	0.19
Random Forest	0.7577	0.7236	0.8235	0.7367	0.7170	2.78
KNN	0.7435	0.7026	0.7912	0.7206	0.6948	0.14
Decision Tree	0.7082	0.6682	0.7516	0.6851	0.6719	0.12

Step 3 — Tune the Best Model with GridSearchCV

```
# ■ GridSearch Tuning ■
# From the leaderboard, suppose Random Forest wins
# Tune its key hyperparameters
param_grid = {
    'n_estimators' : [50, 100, 200],
    'max_depth' : [None, 5, 10],
    'min_samples_leaf': [1, 3, 5],
}
# 3x3x3 = 27 combos x 5-fold = 135 fits – runs in ~1 minute
```

```
gs = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid, cv=cv, scoring='accuracy',
    n_jobs=-1, verbose=0
)
gs.fit(X, y)
print(f'Best params : {gs.best_params_}')
print(f'Best CV acc : {gs.best_score_:.4f}')
# Compare tuned vs default
default_rf = RandomForestClassifier(n_estimators=100, random_state=42)
default_scores = cross_val_score(default_rf, X, y, cv=cv, scoring='accuracy')
tuned_scores = cross_val_score(gs.best_estimator_, X, y, cv=cv, scoring='accuracy')
print(f'\nDefault RF accuracy: {default_scores.mean():.4f} ± {default_scores.std():.4f}')
print(f'Tuned RF accuracy: {tuned_scores.mean():.4f} ± {tuned_scores.std():.4f}')
```

```
Best params : {'max_depth': 5, 'min_samples_leaf': 3, 'n_estimators': 50}
Best CV acc : 0.7759
```

```
Default RF accuracy: 0.7577 ± 0.0266
Tuned RF accuracy: 0.7759 ± 0.0367
```

Step 4 — Final Evaluation + Confusion Matrix Dashboard

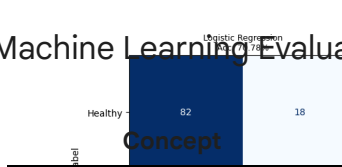
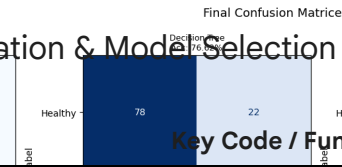
```

# ■ Final Evaluation Dashboard ■
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# One final hold-out test – we only do this ONCE
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))
Final_models = {
    'Logistic Regression': Pipeline([('sc', StandardScaler()), ('m', LogisticRegression())],
    'Decision Tree' : DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest' : gs.best_estimator_, # Tuned version
    'KNN' : Pipeline([('sc', StandardScaler()), ('m', KNeighborsClassifier(n_neighbors=5))],
}
final_results = []
for ax, (name, model) in zip(axes, Final_models.items()):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    ConfusionMatrixDisplay(cm, display_labels=['Healthy', 'Diabetic']).plot(
        ax=ax, cmap='Blues', colorbar=False)
    ax.set_title(f'{name}\nAcc: {acc:.2%}', fontsize=9)
    final_results.append({'Model':name, 'Test Accuracy':f'{acc:.2%}'})
plt.suptitle('Final Confusion Matrices – All Models', fontsize=13, y=1.02)
plt.tight_layout(); plt.show()
# Final report for best model
best_model = Final_models['Random Forest']
print('\n=== FINAL REPORT: Random Forest (Tuned) ===')
print(classification_report(y_test, best_model.predict(X_test),
    target_names=['Healthy', 'Diabetic']))

```

Machine Learning Evaluation & Model Selection – Quick Cheat Sheet

Final Confusion Matrices — All Models		
		
Concept	Key Code / Function	
K-Means Clustering	<code>KMeans(n_clusters=K, n_init=10)</code>	
Elbow Method	<code>km.inertia_</code> for K in <code>range(2,11)</code>	
Silhouette Score	<code>silhouette_score(X, labels)</code>	
Cluster Visualisation	<code>plt.scatter(X[:,0], X[:,1], c=labels)</code>	
Correlation Matrix	<code>df.corr()</code>	
Correlation Heatmap	<code>sns.heatmap(df.corr(), annot=True)</code>	
Outlier Detection (IQR)	<code>Q1 - 1.5*IQR</code> , <code>Q3 + 1.5*IQR</code>	
Outlier Capping	<code>df[col].clip(lower, upper)</code>	
Outlier Removal	<code>df[(df[col]>=lower)&(df[col]<=upper)]</code>	
Training vs Test Error	<code>model.score(X_train,y_train)</code>	154
	<code>model.score(X_test,y_test)</code>	4
Cross Validation	<code>cross_val_score(model,X,y,cv=5)</code>	
Stratified Cross Validation	<code>StratifiedKFold(n_splits=5)</code>	

Always apply **StandardScaler**Look for the **bend (elbow)** wrRange: **0 to 1** → Higher = Bett

Color points by cluster, mark c

Shows feature relationships; c

Strong positive = high positive

Detect outliers using lower an

Winsorizing — reduce outlier i

Remove extreme values if they

Large gap = Overfitting

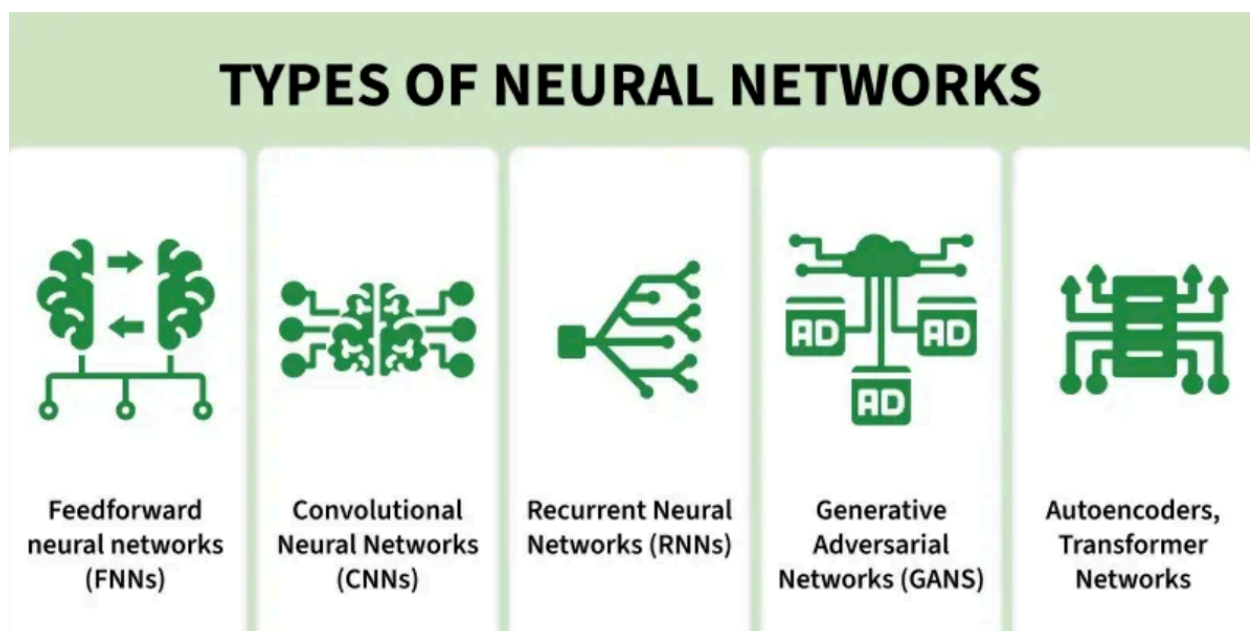
More reliable than a single tra

Maintain class balance in clas

Introduction to Neural Networks

Neural networks power modern AI — from image recognition to large language models. Today you will understand the brain analogy, learn how a network learns step by step, then build your first real neural network in Keras targeting 97%+ accuracy on MNIST. No heavy maths today — intuition and implementation is the goal.

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making



Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

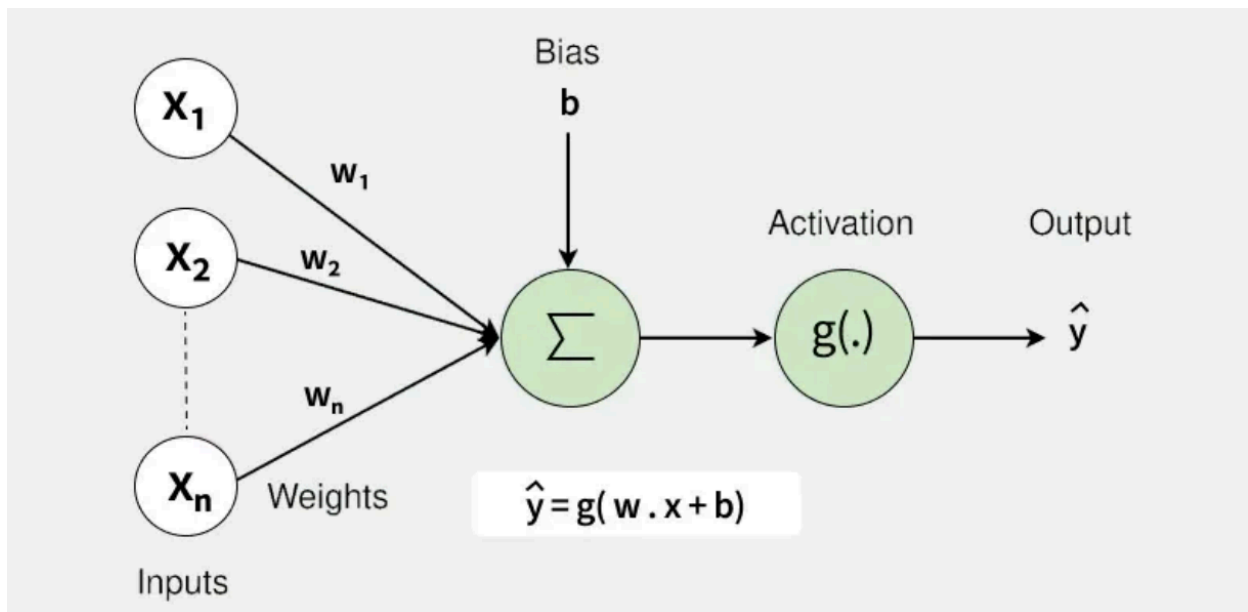
Neurons: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.

Connections: Links between neurons that carry information, regulated by weights and biases.

Weights and Biases: These parameters determine the strength and influence of connections.

Propagation Functions: Mechanisms that help process and transfer data across layers of neurons.

Learning Rule: The method that adjusts weights and biases over time to improve accuracy.



The Human Brain Analogy

Biological Neurons — How Your Brain Works

Your brain contains about 86 billion neurons. Each neuron is a tiny cell that receives electrical signals from other neurons through connections called synapses. When enough signals arrive, the neuron fires — it sends its own signal forward to the next layer of neurons. This chain of firing neurons is how you recognise a face, read these words, or remember a name.

No single neuron understands a face. Thousands of neurons work together — early ones detect edges, middle ones detect shapes, later ones detect eyes and noses. Recognition emerges from cooperation. Artificial neural networks copy this exact idea.

From Neuron to Artificial Neuron

The Artificial Neuron — Step by Step

Step 1 — Receive inputs: Numbers arrive as inputs. For an image, these are pixel values. For a house price model, they might be square footage and location.

Step 2 — Multiply by weights: Each input is multiplied by a weight — a number the network has learned. High weight = this input matters a lot. Low weight = this input barely matters.

Step 3 — Sum everything: Add all weighted inputs together, then add one more learnable number called a bias (shifts the result up or down).

Step 4 — Apply activation function: Pass the sum through a function that squashes it into a useful range and adds non-linearity.

Step 5 — Output: One number leaves this neuron and goes to the next layer.

The Neuron Formula

$\text{output} = \text{activation}(w_1x_1 + w_2x_2 + \dots + w_nx_n + \text{bias})$

In compact matrix form (how computers actually compute it):

$\text{output} = \text{activation}(W \cdot x + b)$

W is a matrix of weights, x is the input vector, b is the bias vector. The dot product $W \cdot x$ computes all weighted sums at once using matrix multiplication — fast, efficient, and the same on a GPU with millions of neurons.

Weights and Bias — What the Network Actually Learns

When we say a neural network learns, exactly two things change: weights and biases. The architecture (number of layers, neurons per layer) is fixed before training. Training is the process of finding the right values for every weight and bias.

Parameter	What it does	Simple analogy
Weight (w)	How much this input influences the output	Volume knob on each input
Bias (b)	Shifts the activation up or down regardless of input	Thermostat baseline offset
Activation	Non-linear squash function applied after the sum	The neuron's on/off decision

Why Do We Need Activation Functions?

The Non-Linearity Problem

If you chain 10 layers of pure matrix multiplications (no activation functions), the entire network is mathematically equivalent to a single matrix multiplication. Depth gives you nothing. You can replace 10 layers with 1 and get identical results.

Activation functions break this by introducing non-linearity. With non-linear activations, a network with enough neurons can approximate any continuous function — this is the Universal Approximation Theorem, the mathematical foundation of deep learning.

Activation Functions — Intuition First

Function	Formula	Output Range	Plain English	Use For
ReLU	$\max(0, x)$	0 to ∞	If positive, pass it. If negative, output 0.	Hidden layers — default
Sigmoid	$1 / (1 + e^{-x})$	0 to 1	Squeezes any number into a probability.	Binary output
Softmax	$e^{x_i} / \sum e^{x_j}$	0 to 1 (sums to 1)	Converts scores to class probabilities.	Multi-class output
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	-1 to +1	Like sigmoid but zero-centred.	Hidden layers
Leaky ReLU	$\max(0.01x, x)$	$-\infty$ to $+\infty$	Like ReLU but small leak for negatives.	Fixes dead neurons

ReLU is the default choice for hidden layers today. It is simple, fast, and avoids the vanishing gradient problem that plagued older sigmoid-only networks. Use Softmax on the output layer for multi-class problems — it ensures all class probabilities add up to 1.0.

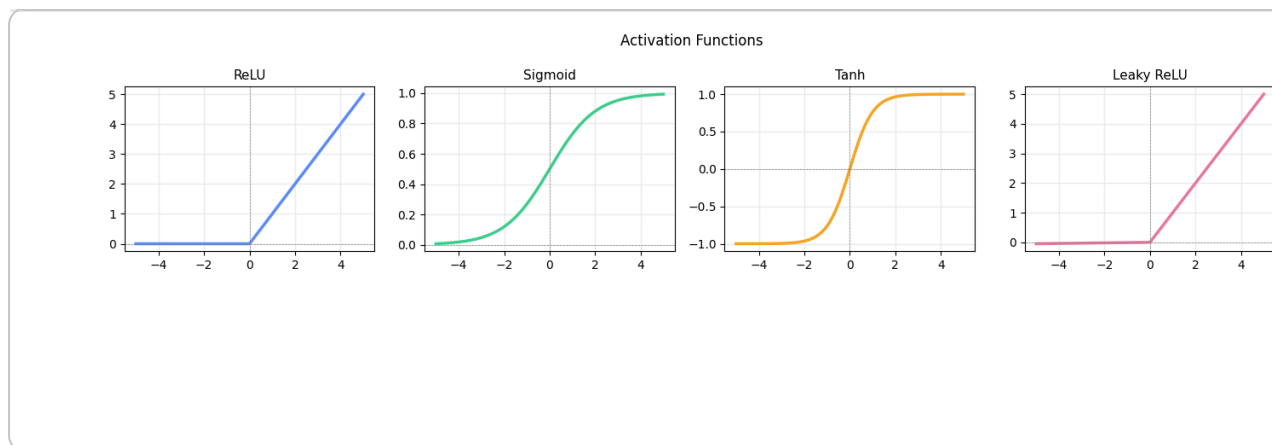
```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-5, 5, 300)

# ■ Define activation functions

relu = lambda x: np.maximum(0, x)
sigmoid = lambda x: 1 / (1 + np.exp(-x))
tanh_fn = np.tanh
leaky = lambda x: np.where(x > 0, x, 0.01 * x)

fig, axes = plt.subplots(1, 4, figsize=(14, 3))
fns = [relu, sigmoid, tanh_fn, leaky]
names = ['ReLU', 'Sigmoid', 'Tanh', 'Leaky ReLU']
cols = ['#5B8FF9', '#3ECF8E', '#F5A623', '#E879A0']

for ax, fn, name, col in zip(axes, fns, names, cols):
    ax.plot(x, fn(x), color=col, linewidth=2.5)
    ax.axhline(0, color='gray', lw=0.5, ls='--') # x-axis
    ax.axvline(0, color='gray', lw=0.5, ls='--') # y-axis
    ax.set_title(name, fontsize=11)
    ax.grid(alpha=0.2)
plt.suptitle('Activation Functions', fontsize=12)
plt.tight_layout(); plt.show()
```



Building a Neural Network

The Three Types of Layers

Every neural network has the same basic structure — data enters at the left, flows through the middle, and exits at the right:

Layer	What it does	MNIST Example (28×28 digit image)
Input Layer	Holds raw input data — no computation, just receives values	784 neurons — one per pixel in the flattened image
Hidden Layers	Learn patterns — each layer learns increasingly complex features	Layer 1: 128 neurons detect strokes. Layer 2: 64 neurons detect curves
Output Layer	Produces the final prediction — size equals number of classes	10 neurons — one per digit 0 through 9

start with 1-2 hidden layers. Add more only if the model cannot fit the data. More layers = more expressive but harder to train and easier to overfit.

The Forward Pass — Data Flowing Through

Step by Step: Classifying the Digit '7'

Input: The 28×28 image is flattened to 784 numbers (pixel values 0.0–1.0 after normalisation).

Hidden Layer 1 (128 neurons): Each neuron computes $W \cdot x + b$ then applies ReLU.

Output: 128 numbers encoding learned features like diagonal strokes and horizontal bars.

Hidden Layer 2 (64 neurons): Receives the 128 features and combines them into 64 higher-level representations — partial shapes, digit-like structures.

Output Layer (10 neurons): Produces 10 raw scores. Softmax converts these to probabilities that sum to 1.0.

Prediction: [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.90, 0.02, 0.01] → highest at index 7 → predicted digit is 7. ✓

Loss Function — Measuring How Wrong We Are

After the forward pass, we compare the prediction to the true label. The loss function converts the error into a single number. The goal of training is to minimise this number.

Loss Function	When to Use	Intuition
<code>sparse_categorical_crossentropy</code>	Multi-class, integer labels (0,1,2...)	Penalises confident wrong answers harshly
<code>categorical_crossentropy</code>	Multi-class, one-hot labels ([0,1,0,0...])	Same maths, different label format
<code>binary_crossentropy</code>	Two classes only (spam/not spam)	Special case of cross-entropy for 2 classes
<code>mean_squared_error</code>	Regression — continuous output	Average squared distance from target

Cross-entropy is large when the model confidently predicts the wrong class, and near zero when it confidently predicts the correct class. This is exactly the behaviour we want — punish arrogant mistakes.

Backpropagation — How the Network Corrects Its Mistakes

Backprop in Plain English

After computing the loss, we need to adjust weights to reduce it. Backpropagation answers: how much did each weight contribute to this mistake?

The Chain Rule: Calculus tells us how to compute the gradient of the loss with respect to every single weight in the network — even weights deep in early layers. We chain the derivatives layer by layer, backward from the output.

The Update: Once we have the gradient for each weight, we nudge it slightly in the direction that reduces the loss: $\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$

Repeat: Do this for every batch of training examples, every epoch. The network gets a little better each time until loss plateaus.

$$\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$$

If the gradient is positive (increasing the weight increases the loss), we decrease the weight. If negative, we increase it. The learning rate controls the step size — too large and we overshoot, too small and training takes forever. Adam optimiser handles this automatically.

Gradient Descent Variants

Variant	Description	Pros / Cons
Batch GD	Compute gradient over entire dataset, then update	Stable but impractically slow for large data
Stochastic GD (SGD)	Compute gradient over 1 sample, update immediately	Fast updates but very noisy — oscillates a lot
Mini-batch GD	Compute gradient over small batch (32–128 samples)	Best of both — the standard in practice
Adam	Adaptive per-weight learning rates + momentum tracking	Default choice — fast, stable, robust
RMSprop	Adapts learning rate based on recent gradient magnitudes	Good for RNNs and non-stationary problems

Epochs and Batch Size

Term	Definition	Practical Guidance
Epoch	One complete pass through all training data	Use 20–50 with EarlyStopping
Batch Size	Samples processed before one weight update	32 or 128 — powers of 2 for GPU efficiency
Iteration	One batch = one forward + backward pass + update	Iterations per epoch = $N / \text{batch_size}$
Learning Rate	Step size for weight updates (default 0.001 with Adam)	Rarely need to change with Adam

Overfitting vs Underfitting — What the Loss Curves Tell You

Healthy training: Both train loss and val loss decrease and run close together. Model is learning genuine patterns.

Overfitting: Train loss keeps dropping but val loss starts rising. Model has memorised training data. Fix: add Dropout, reduce model size, get more data.

Underfitting: Both losses are high and barely decreasing. Model is too simple. Fix: add more neurons, more layers, train longer

Your First Neural Network with Keras

What is Keras?

Keras is a high-level Python library built on top of TensorFlow. It lets you build, train, and evaluate neural networks using simple, readable code — without writing the backpropagation algorithm or GPU matrix operations yourself.

The Four Keras Steps

1. Build: Stack layers with `keras.Sequential()`. Each Dense layer is a fully connected layer where every neuron connects to every neuron in the next layer.

2. Compile: Choose your loss function (what to minimise) and optimiser (how to minimise it). This is where you wire up the learning machinery.
3. Fit: Call `model.fit()` with your data. Keras runs the training loop — forward pass, loss, backprop, update — for every batch and every epoch.
4. Evaluate: Call `model.evaluate()` on test data you have never trained on. This is your true accuracy number.

Building the Model — Annotated Line by Line

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# ■ Step 1: Load MNIST
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# ■ Step 2: Preprocess
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
print(f'Train: {X_train.shape} Test: {X_test.shape}')

# ■ Step 3: Build the model
model = keras.Sequential([
    keras.Input(shape=(784,)),

    layers.Dense(234, activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(10, activation='softmax'),
])
model.summary()
```

[Show hidden output](#)

Compile and Train

```
# ■ Step 4: Compile
model.compile(
```



```
# Loss
ax2.plot(history.history['loss'], label='Train', color='#F5A623', lw=2)
ax2.plot(history.history['val_loss'], label='Val', color='#E879A0', lw=2)
ax2.set(title='Loss', xlabel='Epoch', ylabel='Loss')
ax2.legend(); ax2.grid(alpha=0.3)
plt.tight_layout(); plt.show()

# Reading curves:
# GOOD: both curves decline and stay close together
# OVERFIT: train keeps dropping, val starts rising
# UNDERFIT: both stay high – model too simple or not trained enough
```

Show hidden output

Lab — MNIST Classifier: Target 97%+

Build, train, and evaluate a neural network on the full MNIST dataset. Visualise predictions, inspect the confusion matrix, examine misclassified digits, and experiment with different architectures to understand how design choices affect accuracy.

Complete Lab Code

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import confusion_matrix, classification_report

# Load and preprocess MNIST
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 784).astype('float32') / 255.0
X_test = X_test.reshape(-1, 784).astype('float32') / 255.0

# Visualise a sample of training digits
fig, axes = plt.subplots(2, 10, figsize=(14, 3))
for d in range(10):
    for r, offset in enumerate([0, 1]):
        idx = np.where(y_train == d)[0][offset]
        axes[r, d].imshow(X_train[idx].reshape(28,28), cmap='gray')
        axes[r, d].set_title(str(d), fontsize=8); axes[r, d].axis('off')
plt.suptitle('MNIST: 2 samples per digit'); plt.tight_layout(); plt.show()

# Build model as a reusable function
def make_model(h1=256, h2=128, drop=0.3):
    m = keras.Sequential([
        keras.Input(shape=(784,)),
        layers.Dense(h1, activation='relu'),
        layers.Dropout(drop),
        layers.Dense(h2, activation='relu'),
        layers.Dropout(drop),
```

```

layers.Dense(10, activation='softmax'),
])
m.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
return m
model = make_model()
model.summary()

```

[Show hidden output](#)

```

# ■ Train
history = model.fit(
X_train, y_train, epochs=30, batch_size=128,
validation_split=0.1,
callbacks=[EarlyStopping(patience=5, restore_best_weights=True)],
verbose=1)
# ■ Evaluate
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {acc*100:.2f}%')
# ■ Confusion matrix
y_pred = np.argmax(model.predict(X_test), axis=1)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=range(10), yticklabels=range(10))
plt.title(f'Confusion Matrix - Test Accuracy {acc*100:.1f}%')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.tight_layout(); plt.show()
# Diagonal = correctly classified. Off-diagonal = confused pairs.
# Common confusions: 4 vs 9, 3 vs 8, 5 vs 6
print(classification_report(y_test, y_pred))
# ■ Show misclassified images
wrong = np.where(y_pred != y_test)[0]
print(f'Misclassified: {len(wrong)} out of {len(y_test)}')
fig, axes = plt.subplots(3, 8, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    if i < len(wrong):
        idx = wrong[i]
        ax.imshow(X_test[idx].reshape(28,28), cmap='gray')
        ax.set_title(f'T:{y_test[idx]} P:{y_pred[idx]}',
            color='red', fontsize=7)
        ax.axis('off')
plt.suptitle('Misclassified Digits (T=True, P=Predicted)')
plt.tight_layout(); plt.show()
# ■ Architecture comparison
configs = [
(64, 32, 0.0, 'Small / no dropout'),
(128, 64, 0.2, 'Medium / light dropout'),
(256, 128, 0.3, 'Large / moderate dropout'),
]
for h1, h2, dr, label in configs:
    m = make_model(h1, h2, dr)
    m.fit(X_train, y_train, epochs=15, batch_size=128,

```

```

validation_split=0.1, verbose=0,
callbacks=[EarlyStopping(patience=3, restore_best_weights=True)])
_, a = m.evaluate(X_test, y_test, verbose=0)
print(f'{label:35s} Accuracy: {a*100:.2f}%')

```

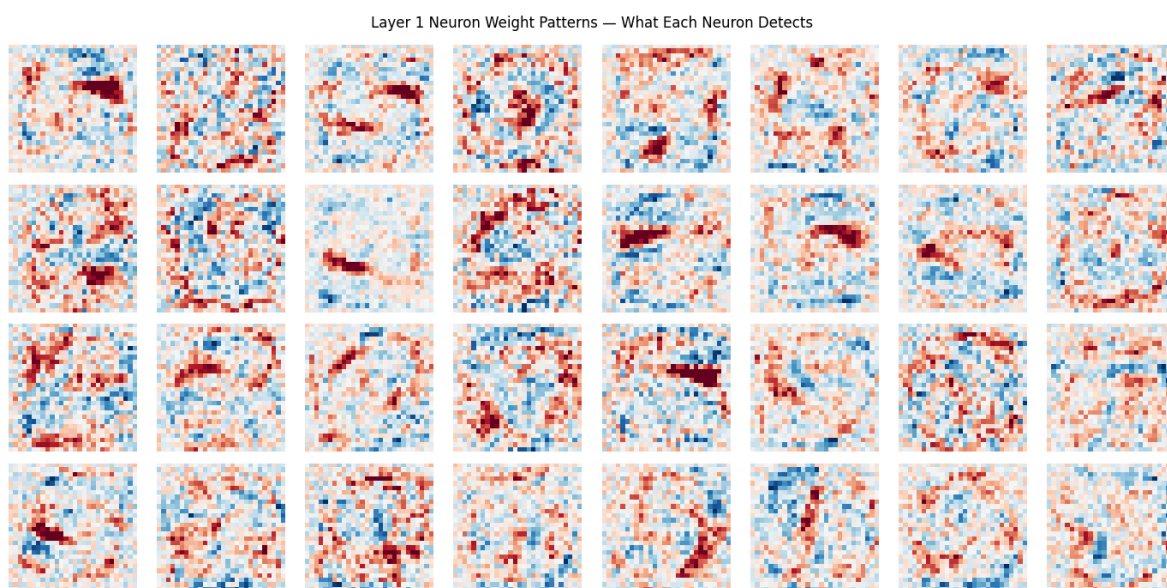
[Show hidden output](#)

Bonus: Visualise What Neurons Learn

```

# ■ Each Layer 1 neuron has 784 weights (one per pixel) ■
# Reshape to 28x28 to see which pixel patterns each neuron responds to
weights = model.layers[0].get_weights()[0] # shape: (784, 256)
fig, axes = plt.subplots(4, 8, figsize=(14, 7))
for i, ax in enumerate(axes.flat):
    w = weights[:, i].reshape(28, 28)
    ax.imshow(w, cmap='RdBu', vmin=-w.max(), vmax=w.max())
    ax.axis('off')
plt.suptitle('Layer 1 Neuron Weight Patterns — What Each Neuron Detects')
plt.tight_layout(); plt.show()
# Blue = neuron activated strongly by bright pixel here
# Red = neuron suppressed by bright pixel here

```



Theory Deep Dive

The Vanishing Gradient Problem

During backpropagation, gradients flow backward through every layer using the chain rule. Each layer multiplies the gradient by a number. With sigmoid activations, that number is always between 0 and 0.25 (because the sigmoid derivative never exceeds 0.25).

With 10 layers: $0.25^{10} = 0.00000095$ — essentially zero. Weights in early layers receive a gradient so small they barely update. The network's front end stops learning entirely. This was the main barrier to training deep networks before 2010

Solutions to Vanishing Gradient

ReLU Activation: Gradient of ReLU is 1 for positive inputs. No shrinking. This single change enabled deep networks to be trained reliably.

Batch Normalisation: Normalises layer inputs during training so gradients stay in a healthy range throughout the network depth.

Residual Connections (ResNet): Skip connections allow gradients to flow directly backward bypassing layers, preventing the gradient from being multiplied to zero.

The Universal Approximation Theorem

A neural network with one hidden layer and enough neurons can approximate any continuous function to any desired accuracy. This is the mathematical guarantee that neural networks are not limited — they can in principle learn any mapping from inputs to outputs.

In practice: one wide layer requires exponentially many neurons for complex functions. Deep networks (many layers, moderate width) achieve the same using hierarchical composition — each layer builds on the previous, reusing simpler patterns to represent complex ones. This is why depth matters more than width.

Parameter Count — Understanding model.summary()

Layer	Input Size	Neurons	Parameters Formula	Count
Dense (128)	784	128	$784 \times 128 + 128 \text{ bias}$	100,480
Dropout (0.2)	128	128	0 — no learnable params	0
Dense (64)	128	64	$128 \times 64 + 64 \text{ bias}$	8,256
Dropout (0.2)	64	64	0 — no learnable params	0
Dense (10)	64	10	$64 \times 10 + 10 \text{ bias}$	650
TOTAL				109,386

Regularisation Techniques

Technique	How It Works
Dropout	Randomly sets a fraction of neurons to zero during each training step, reducing overfitting.
L2 / Weight Decay	Adds a penalty for large weights to the loss function, encouraging simpler models.
Early Stopping	Stops training when validation loss stops improving, preventing overfitting.
Batch Normalization	Normalizes layer inputs, stabilizes learning, and often speeds up training.
Data Augmentation	Artificially expands the dataset using transformations like flips, rotations, zooms, etc.
More Data	The most effective regularization technique; provides the model with more examples to learn from.

Hyperparameter Quick Reference

Hyperparameter	Start Here	Too Low →	Too High →
Learning Rate	0.001 (Adam)	Training very slow	Loss explodes or oscillates
Batch Size	128	Slow, noisy updates	Less generalisation, more memory
Neurons per Layer	64–256	Underfitting	Overfitting, slower training
Dropout Rate	0.2–0.3	Overfitting persists	Underfitting, slow convergence
Number of Layers	2 hidden	Underfitting	Vanishing gradients, overfitting

Neural Networks vs Classical ML

Use Case	Best Choice	Why
Tabular data, small (<10k rows)	Random Forest / XGBoost	NNs overfit; trees are robust and fast
Tabular data, large (>100k rows)	Try both and compare	NNs start to match trees with more data
Images, audio, video	Neural Network (CNN)	Traditional ML cannot handle raw pixels well
Text / NLP	Neural Network (Transformer)	Sequence data is a NN strength
Need model explanation	Decision Tree / Logistic Reg	NNs are black boxes
Limited compute / time	Naive Bayes / Logistic Reg	NNs need GPU and significant training time

Feature Engineering & Data Preprocessing

What is Feature Engineering?

The art of turning raw data into signals a model can learn from.

Raw data rarely comes in the perfect shape for a machine learning model. Feature engineering is the process of using domain knowledge to create, transform, or select the input variables (features) that best represent the underlying problem — so the model can learn more easily and accurately.

HOW DO WE GET RAW DATA FOR MACHINE LEARNING?

Machine Learning models require data to learn patterns and make predictions. The first step in any ML project is collecting raw data from different sources.

1. User-Generated Data

- Online registration forms
- Job application forms
- Surveys and questionnaires
- Customer feedback forms
- Product reviews and ratings
- Social media interactions

Example: Netflix collects movie ratings and watch history from users.

2. Application Usage Data Data generated when users interact with applications.

- Clicks and page visits
- Search history
- Time spent on pages
- Purchase history
- Login activity
- User behavior tracking

Example: Amazon records products viewed, searched, and purchased.

3. Sensors and IoT Devices Physical devices continuously generate data.

- Smart watches
- Fitness trackers
- GPS devices
- Temperature sensors
- Traffic cameras

Example: A fitness tracker records heart rate and step count data.

4. Public Datasets Many organizations provide datasets for research and learning.

Sources:

- Kaggle
- UCI Machine Learning Repository
- Google Dataset Search
- Government Open Data Portals

Example: The Iris Flower Dataset is widely used for classification tasks.

5. Company Databases Organizations store large amounts of historical business data.

- Customer records
- Sales transactions
- Inventory data
- Banking records
- Healthcare records

Example: Banks use historical loan records to predict loan defaults.

6. APIs (Application Programming Interfaces) APIs provide structured access to data.

- Weather APIs
- Stock Market APIs
- News APIs
- Social Media APIs
- Sports APIs

Example: A weather prediction application collects temperature data through weather APIs.

7. Web Scraping Data can be collected from websites using scraping tools.

- Product prices
- News articles
- Hotel reviews
- Job postings
- Real estate listings

Example: House price data can be collected from real estate websites for price prediction models.

8. Beta Testing and Pilot Programs Before an application is fully launched, selected users test it.

Data collected:

- Feature usage
- Bugs and errors
- User feedback
- Performance metrics

Example: A new mobile application collects feedback from beta users before public release.

MACHINE LEARNING DATA PIPELINE

Data Sources



Raw Data Collection



Data Cleaning



Feature Engineering



Training Dataset



Machine Learning Model

Real-world example: date of birth

A raw date-of-birth column is nearly useless as-is. Here's how to extract useful features from it:

Raw column	Extracted features	Why useful?
date_of_birth	age (in years)	Direct numeric signal
date_of_birth	generation (Boomer / Gen-Z)	Captures cohort behaviour
date_of_birth	birth_month (1-12)	Seasonal patterns
date_of_birth	birth_weekday (Mon-Sun)	Day-of-week patterns

feature extraction from dates

```
import pandas as pd
# Sample data
df = pd.DataFrame({'dob': ['1995-03-15', '1988-11-02', '2001-07-20']})
# Convert string to datetime first
df['dob'] = pd.to_datetime(df['dob'])
# Extract useful features
df['age'] = 2024 - df['dob'].dt.year
df['birth_month'] = df['dob'].dt.month
df['birth_day'] = df['dob'].dt.day_name() # e.g. 'Monday'
# Bin age into generations
bins = [0, 28, 44, 60, 100]
labels = ['Gen-Z', 'Millennial', 'Gen-X', 'Boomer']
df['generation'] = pd.cut(df['age'], bins=bins, labels=labels)
print(df[['age', 'birth_month', 'birth_day', 'generation']])
```

```
   age  birth_month  birth_day  generation
0   29             3  Wednesday  Millennial
1   36            11  Wednesday  Millennial
2   23             7    Friday    Gen-Z
```

Other common feature engineering tricks

Feature Engineering Technique	Example	
Combine Columns (Ratio Features)	<code>price_per_sqft = price / area</code>	Creates a new feature
Log Transformation	<code>log(salary)</code>	Reduces the impact of outliers
Interaction Terms	<code>is_weekend AND is_sale</code>	Captures combined effects
Binning / Discretisation	<code>age: 0-18 → Minor, 19-65 → Adult, 65+ → Senior</code>	Converts continuous variables into categories
Polynomial Features	<code>x, x², x³</code>	Allows linear models to capture non-linear relationships

Handling Categorical Data

Models understand numbers, not words — so we have to translate. Most real-world datasets contain text categories: city names, product types, yes/no flags. ML

algorithms can't do maths on the word 'Mumbai', so we must encode categories as numbers first.

Method 1 — Label Encoding

Replaces each category with an integer. Simple and compact, but it implies an ordering ($2 > 1$

0), which is only correct when the category genuinely has an order (e.g. Small < Medium < Large).

Original value	Encoded value
Yes	1
No	0
Small	0
Medium	1
Large	2

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
df['purchased'] = ['Yes', 'No', 'Yes']
df['size'] = ['Small', 'Medium', 'Large']
le = LabelEncoder()
df['purchased_enc'] = le.fit_transform(df['purchased'])
size_order = {'Small': 0, 'Medium': 1, 'Large': 2}
df['size_enc'] = df['size'].map(size_order)
print(df[['purchased', 'purchased_enc', 'size', 'size_enc']])
```

	purchased	purchased_enc	size	size_enc
0	Yes	1	Small	0
1	No	0	Medium	1
2	Yes	1	Large	2

Method 2 — One-Hot Encoding

Creates one new binary (0/1) column per category. No false ordering is implied. This is the standard approach for nominal categories (no natural order).

City	Delhi	Mumbai	Chennai
Delhi	1	0	0
Mumbai	0	1	0

```
import pandas as pd
df = pd.DataFrame({'city': ['Delhi', 'Mumbai', 'Chennai', 'Delhi']})
df_encoded = pd.get_dummies(df, columns=['city'], dtype=int)
print(df_encoded)

print("-----")
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(df[['city']])

print(encoded)
```

```
city_Chennai  city_Delhi  city_Mumbai
0             0           1             0
1             0           0             1
2             1           0             0
3             0           1             0
```

```
-----
[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

When to use which?

Situation	Use	Example
Categories have a natural order	Label Encoding	Small / Medium / Large
Binary yes/no column	Label Encoding	Purchased: Yes/No
No order, few categories (<10)	One-Hot Encoding	City: Delhi/Mumbai/Chennai
No order, many categories (10+)	Target Encoding	Postcode, product SKU

Handling Missing Data

Real-world data is always incomplete — here's how to deal with it. Missing values are one of the most common data quality problems. If you pass a DataFrame with NaN values directly into sklearn, it will throw an error. You must decide: fill the gap, or drop the row?

Why does data go missing?

MCAR

Missing Completely At Random A sensor randomly fails. Nothing systematic — safe to drop rows.

MAR

Missing At Random Older people skip income questions more than younger. Pattern exists but is explainable.

MNAR

Missing Not At Random High earners hide their salary. The very fact of missing-ness carries signal — hardest to handle.

Step 1 — Diagnose missing values

```
import pandas as pd
import numpy as np

# Create a sample DataFrame with missing values
df = pd.DataFrame({
    'feature_A': [1, 2, np.nan, 4, 5],
    'feature_B': ['X', 'Y', 'Z', np.nan, 'Y'],
    'feature_C': [10.5, np.nan, 12.3, 14.0, np.nan]
})

print('Missing values count per column:')
print(df.isnull().sum())
print('\n')
print('Percentage missing per column:')
print(df.isnull().mean() * 100)

# !pip install missingno
# import missingno as msno
# msno.matrix(df)
```

[Show hidden output](#)

Step 2 — Fill or drop?

Scenario	Recommended Action
< 5% of rows missing in a column	Drop those rows
	Data loss

Scenario	Recommended Action	
5–30% missing, Numeric Column	Fill with Mean (if normally distributed) or Median (if skewed)	Preserv
5–30% missing, Categorical Column	Fill with Mode (most frequent value)	Maintain
> 30% missing in a column	Consider dropping the entire column	Too muc
MNAR (Missing Not At Random)	Create a binary indicator feature (e.g., <code>was_salary_missing</code>)	Missing

SimpleImputer — filling missing values

```
from sklearn.impute import SimpleImputer
import numpy as np
numeric_cols = ['feature_A', 'feature_C']
num_imputer = SimpleImputer(strategy='median')

df[numeric_cols] = num_imputer.fit_transform(df[numeric_cols])

categorical_cols = ['feature_B']
cat_imputer = SimpleImputer(strategy='most_frequent')

df[categorical_cols] = cat_imputer.fit_transform(df[categorical_cols])

print('Total missing values after imputation:', df.isnull().sum().sum()) # 5
print('\nDataFrame after imputation:')
print(df)
```

Total missing values after imputation: 0

DataFrame after imputation:

	feature_A	feature_B	feature_C
0	1.0	X	10.5
1	2.0	Y	12.3
2	3.0	Z	12.3
3	4.0	Y	14.0
4	5.0	Y	12.3

Outlier handling

Outliers are extreme values that can distort model training. Two common detection methods:

```
# Method 1: IQR (interquartile range) – robust to distribution shape
# Using 'feature_A' as a numerical column for demonstration
Q1 = df['feature_A'].quantile(0.25)
Q3 = df['feature_A'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Keep only rows within bounds
df_clean_iqr = df[(df['feature_A'] >= lower) & (df['feature_A'] <= upper)]
print("DataFrame after IQR outlier removal on 'feature_A':")
```

```

print(df_clean_iqr)
print('\n')

# Method 2: Z-score – works well for normally distributed data
from scipy import stats
# Using 'feature_A' for demonstration
z_scores = stats.zscore(df['feature_A'])
df_clean_zscore = df[abs(z_scores) < 3] # remove rows where |z| > 3
print("DataFrame after Z-score outlier removal on 'feature_A' (threshold 3):")
print(df_clean_zscore)

```

DataFrame after IQR outlier removal on 'feature_A':

	feature_A	feature_B	feature_C
0	1.0	X	10.5
1	2.0	Y	12.3
2	3.0	Z	12.3
3	4.0	W	14.0
4	5.0	W	12.3

DataFrame after Z-score outlier removal on 'feature_A' (threshold 3):

	feature_A	feature_B	feature_C
0	1.0	X	10.5
1	2.0	Y	12.3
2	3.0	Z	12.3
3	4.0	W	14.0
4	5.0	W	12.3

Mean vs median

for imputation Use mean only when your data is roughly symmetric (no big outliers). Use median when data is skewed (income, house prices, etc.) — the median is not pulled by extreme values the way the mean is.

sklearn Pipelines

Chain every preprocessing step into one neat, leak-proof object. A Pipeline bundles preprocessing steps and a final estimator into a single object. You call `.fit()` once and everything runs in sequence — cleaning, scaling, then training. This prevents data leakage and makes deployment far simpler.

What is data leakage?

Data leakage — the silent model killer

Leakage happens when information from the test set 'leaks' into your training.

Example: if you calculate mean salary on the WHOLE dataset before splitting, your test rows have already influenced the imputer. The model looks great in evaluation but

fails in production. Pipelines prevent this by fitting only on training data and applying the same transform to test data.

Building a 3-step pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

# Assuming 'df' is the DataFrame from previous steps
# For demonstration, let's define X and y from the existing 'df'
X = df[['feature_A', 'feature_C']] # Using numeric features as X
# Create a dummy target variable 'y' for classification example
# e.g., if feature_A is greater than its median, classify as 1, else 0
y = (df['feature_A'] > df['feature_A'].median()).astype(int)

# Step 1: split data BEFORE building the pipeline
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 2: define the pipeline (a list of (name, transformer) tuples)
pipe = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')), # fill NaNs
    ('scaler', StandardScaler()), # scale features
    ('model', LogisticRegression(random_state=42)), # train classifier
])

# Step 3: fit ONLY on training data – one call does everything
pipe.fit(X_train, y_train)

# Step 4: evaluate – the pipeline automatically transforms X_test
# using parameters learned from X_train (no leakage!)
print('Accuracy:', pipe.score(X_test, y_test))

# Predict new data in production – just one call
# For demonstration, create a sample X_new based on df's columns
X_new = pd.DataFrame({
    'feature_A': [6.0, np.nan, 8.0],
    'feature_C': [15.0, 16.0, np.nan]
})
predictions = pipe.predict(X_new)
print('Predictions for new data:', predictions)
```

```
Accuracy: 1.0
Predictions for new data: [1 1 1]
```

Lab — Clean a Messy Dataset

Part A — Create the messy dataset

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
np.random.seed(42)
n = 500
# Generate synthetic HR data with intentional problems
df = pd.DataFrame({
    'age': np.random.randint(22, 60, n).astype(float),
    'salary': np.random.randint(25000, 120000, n).astype(float),
    'dept': np.random.choice(['Eng', 'Sales', 'HR', 'Finance'], n),
    'education': np.random.choice(['HS', 'BSc', 'MSc', 'PhD'], n),
    'years_exp': np.random.randint(0, 35, n).astype(float),
    'left_job': np.random.randint(0, 2, n), # target: 1=left, 0=stayed
})
# Inject missing values (~12% each column)
for col in ['age', 'salary', 'dept']:
    df.loc[df.sample(frac=0.12).index, col] = np.nan
# Inject outliers
df.loc[0, 'salary'] = 999999 # extreme outlier
df.loc[1, 'age'] = 150 # impossible age
print('Shape:', df.shape)
print(df.isnull().sum())
```

```
Shape: (500, 6)
age      60
salary   60
dept     60
education 0
years_exp 0
left_job  0
dtype: int64
```

Part B — Baseline (no cleaning)

```
# Baseline: just drop rows with missing values – very naive
df_base = df.dropna()
X_base = pd.get_dummies(df_base.drop('left_job', axis=1))
y_base = df_base['left_job']
X_tr, X_te, y_tr, y_te = train_test_split(X_base, y_base,
test_size=0.2, random_state=42)
baseline = LogisticRegression(max_iter=200)
baseline.fit(X_tr, y_tr)
base_acc = accuracy_score(y_te, baseline.predict(X_te))
print(f'Baseline accuracy: {base_acc:.3f}')
# Typically around 0.48-0.52 (near random) due to data loss + no scaling
```

```
Baseline accuracy: 0.464
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
```

STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres

```
n_iter_i = _check_optimize_result(
```

Part C — Full pipeline (clean model)

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

# 1. Define column groups
num_cols = ['age', 'salary', 'years_exp']
cat_cols = ['dept', 'education']

# 2. Fix the impossible outlier BEFORE pipeline
# (domain knowledge: age > 100 is impossible)
df['age'] = df['age'].clip(upper=100)
df['salary'] = df['salary'].clip(upper=200000)

# 3. Split BEFORE any fitting
X = df.drop('left_job', axis=1)
y = df['left_job']
X_tr, X_te, y_tr, y_te = train_test_split(X, y,
test_size=0.2, random_state=42)

# 4. Build sub-pipelines
num_pipe = Pipeline([
('impute', SimpleImputer(strategy='median')),
('scale', StandardScaler()),
])
cat_pipe = Pipeline([
('impute', SimpleImputer(strategy='most_frequent')),
('encode', OneHotEncoder(handle_unknown='ignore',
sparse_output=False)),
])

# 5. Combine
preprocessor = ColumnTransformer([
('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols),
])

# 6. Full pipeline
model = Pipeline([
('prep', preprocessor),
('clf', LogisticRegression(max_iter=1000)),
])
model.fit(X_tr, y_tr)
clean_acc = accuracy_score(y_te, model.predict(X_te))
print(f'Clean pipeline accuracy: {clean_acc:.3f}')
print(f'Improvement: +{clean_acc - base_acc:.3f}')
# Expect 0.62-0.70 – a clear improvement over baseline
```

Clean pipeline accuracy: 0.460
Improvement: +-0.004

Part D — Explore results

```
from sklearn.metrics import classification_report, confusion_matrix
y_pred = model.predict(X_te)
# Detailed metrics
print(classification_report(y_te, y_pred,
target_names=['Stayed', 'Left']))
# Confusion matrix
print(confusion_matrix(y_te, y_pred))
# Cross-validate to get a more reliable estimate
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f'CV accuracy: {scores.mean():.3f} +/- {scores.std():.3f}')
```

	precision	recall	f1-score	support
Stayed	0.42	0.33	0.37	48
Left	0.48	0.58	0.53	52
accuracy			0.46	100
macro avg	0.45	0.46	0.45	100
weighted avg	0.45	0.46	0.45	100

```
[[16 32]
 [22 30]]
```

CV accuracy: 0.432 +/- 0.070